



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE ENGENHARIA DE AUTOMAÇÃO E SISTEMAS
CURSO DE GRADUAÇÃO EM ENGENHARIA DE CONTROLE E AUTOMAÇÃO

Aruã Viggiano Souza

Inteligência Artificial de Borda para Leitura Automática de Placas Veiculares

Florianópolis
2026

Aruã Viggiano Souza

Inteligência Artificial de Borda para Leitura Automática de Placas Veiculares

Relatório final da disciplina DAS5511 (Projeto de Fim de Curso) como Trabalho de Conclusão do Curso de Graduação em Engenharia de Controle e Automação da Universidade Federal de Santa Catarina em Florianópolis.

Orientador: Prof. Eric Aislan Antonelo, Dr.

Supervisor: Dennis Kerr Coelho, Me.

Florianópolis

2026

Ficha de identificação da obra

A ficha de identificação é elaborada pelo próprio autor.

Orientações em:

<http://portalbu.ufsc.br/ficha>

Aruã Viggiano Souza

Inteligência Artificial de Borda para Leitura Automática de Placas Veiculares

Esta monografia foi julgada no contexto da disciplina DAS5511 (Projeto de Fim de Curso) e aprovada em sua forma final pelo Curso de Graduação em Engenharia de Controle e Automação

Florianópolis, 26 de junho de 2026.

Prof. Marcelo De Lellis Costa de Oliveira, Dr.
Coordenador do Curso

Banca Examinadora:

[preencher somente após a defesa para a Versão Final da BU]

Prof. Eric Aislan Antonelo, Dr.
Orientador
UFSC/CTC

Dennis Kerr Coelho, Me.
Supervisor
Palmssoft Tecnologia

Prof(a). xxxx, Dr(a).
Avaliador(a)
Instituição xxxx

Prof. xxxx, Dr.
Presidente da Banca
UFSC/CTC/EAS

Dedico este trabalho à minha linda esposa, Bruna, que esteve comigo me apoiando durante toda a minha graduação. Também dedico aos meus amados filhos, Aurora, minha primogênita, e Antônio, ainda por nascer.

AGRADECIMENTOS

Ofereço meus sinceros agradecimentos à meu supervisor e chefe, Dennis Kerr Coelho, que há quatro anos me contratou como estagiário e ajudou a me formar como profissional. Foi a partir de um interesse dele em testar opções baratas de *hardware* para inteligência artificial que surgiu a ideia para o presente trabalho.

Agradeço também à Alexandre Sena e Igor Luna, por me incluírem no projeto do *Aegis Access Control*, do qual surgiu a principal motivação para este trabalho, mesmo eu não tendo nenhuma experiência com gerenciamento de infraestrutura, que é a tarefa que hoje realizo dentro do projeto e me fez aprender muito.

Por fim, agradeço a meu orientador, Prof. Eric Aislan Antonelo, por sua agilidade em responder minhas dúvidas e também por me introduzir no mundo da Inteligência Artificial dentro da universidade, o que foi crucial para que eu iniciasse minha carreira nessa área.

DECLARAÇÃO DE PUBLICIDADE

Florianópolis, 26 de junho de 2026.

Na condição de representante da Palmsoft Tecnologia na qual o presente trabalho foi realizado, declaro não haver ressalvas quanto ao aspecto de sigilo ou propriedade intelectual sobre as informações contidas neste documento, que impeçam a sua publicação por parte da Universidade Federal de Santa Catarina (UFSC) para acesso pelo público em geral, incluindo a sua disponibilização *online* no Repositório Institucional da Biblioteca Universitária da UFSC. Além disso, declaro ciência de que <o/a> autor<a>, na condição de estudante da UFSC, é obrigado a depositar este documento, por se tratar de um Trabalho de Conclusão de Curso, no referido Repositório Institucional, em atendimento à Resolução Normativa n° 126/2019/CUn.

Por estar de acordo com esses termos, subscrevo-me abaixo.

Dennis Kerr Coelho
Palmsoft Tecnologia

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema de leitura automática de placas de veículos (ALPR – *Automatic License Plate Recognition*) com foco em viabilizar uma solução de baixo custo executável em hardware embarcado acessível. O problema surgiu no contexto do produto *Aegis Access Control*, desenvolvido pela Palmsoft Tecnologia, em que câmeras LPR comerciais de alto custo são utilizadas para controle de acesso em condomínios autônomos de aluguel de curto prazo. O objetivo do trabalho é criar e validar um algoritmo de leitura de placas de veículos brasileiras e analisar sua performance ao ser executado na placa Radxa ROCK 3A, equipada com o chip Rockchip RK3568, que conta com uma NPU de 0,8 TOPs. A solução proposta consiste em um *pipeline* de visão computacional estruturado em três etapas: detecção e classificação da placa por meio do modelo YOLO11n-pose, pré-processamento da imagem da placa com correção de perspectiva, binarização adaptativa pelo método de Sauvola e segmentação de caracteres, e classificação dos caracteres extraídos por redes neurais convolucionais especializadas. Uma abordagem alternativa baseada em modelos de extração e classificação simultânea de caracteres também foi desenvolvida e avaliada. O desenvolvimento seguiu o processo CRISP-DM e utilizou o *dataset* público RodoSol-ALPR. Os modelos foram treinados em ambiente de desenvolvimento, exportados para o formato ONNX e convertidos para o formato RKNN com quantização *uint8* para execução na NPU. Os resultados obtidos demonstram acurácia de 91,05% com o *pipeline* de classificação simultânea e de 85,75% com o *pipeline* de caracteres isolados. Na ROCK 3A, o melhor tempo de inferência foi de 464,10 ms com YOLO executando na NPU e o modelo simultâneo na CPU, e de 539,22 ms com todos os modelos do *pipeline* de caracteres isolados na NPU. Os resultados confirmam a viabilidade técnica e econômica da proposta, com redução de custo por unidade de R\$ 5.700–7.500 para valores da ordem de poucas centenas de reais.

Palavras-chave: Leitura automática de placas de veículos. Visão computacional. Redes neurais convolucionais. Hardware embarcado. NPU. YOLO. Controle de acesso.

ABSTRACT

This work presents the development of an Automatic License Plate Recognition (ALPR) system aimed at enabling a low-cost solution deployable on affordable embedded hardware. The problem originated in the context of the *Aegis Access Control* product, developed by Palmsoft Tecnologia, in which high-cost commercial LPR cameras are used for access control in autonomous short-term rental condominiums. The goal of this work is to develop and validate a license plate reading algorithm for Brazilian vehicle plates and to analyze its performance when executed on the Radxa ROCK 3A board, equipped with the Rockchip RK3568 chip, which includes a 0.8 TOPs Neural Processing Unit (NPU). The proposed solution consists of a computer vision pipeline structured in three stages: license plate detection and classification using the YOLO11n-pose model, license plate image preprocessing including perspective correction, adaptive binarization using the Sauvola method, and character segmentation, and classification of the extracted characters using specialized convolutional neural networks. An alternative approach based on simultaneous character extraction and classification models was also developed and evaluated. The development followed the CRISP-DM process and used the public RodoSol-ALPR dataset. The models were trained in a development environment, exported to ONNX format, and converted to RKNN format with uint8 quantization for NPU execution. Results show an accuracy of 91.05% with the simultaneous classification pipeline and 85.75% with the isolated character pipeline. On the ROCK 3A, the best inference time achieved was 464.10 ms with YOLO running on the NPU and the simultaneous model on the CPU, and 539.22 ms with all models of the isolated character pipeline running on the NPU. The results confirm the technical and economic feasibility of the proposal, reducing the per-unit cost from R\$ 5,700–7,500 to the order of a few hundred reais.

Keywords: Automatic license plate recognition. Computer vision. Convolutional neural networks. Embedded hardware. NPU. YOLO. Access control.

LISTA DE FIGURAS

Figura 1 – Fases do processo CRISP-DM (Chapman et al., 2000)	20
Figura 2 – Camada convolucional (IBM Inc., 2024)	23
Figura 3 – Rede neural convolucional para classificação de imagens (Saha, 2018)	24
Figura 4 – Exemplo de detecção de objetos	25
Figura 5 – Diagrama de funcionamento do modelo YOLO (Redmon et al., 2016)	26
Figura 6 – Exemplo de detecção de pontos-chave em um corpo humano	26
Figura 7 – Transformação de perspectiva (TheAILearner, 2023)	28
Figura 8 – Exemplo de separação de regiões por componentes conexos	30
Figura 9 – A NPU. (a) NPU de 8 <i>processing engines</i> ; (b) Fila de uma <i>processing engine</i> (Chen et al., 2020)	31
Figura 10 – Placa Radxa ROCK 3A	36
Figura 11 – Modelo de placa veicular do padrão de 1990	41
Figura 12 – Modelo de placa veicular do padrão de 1990 para veículos de transporte pago	42
Figura 13 – Modelo de placa veicular do padrão Mercosul	42
Figura 14 – Exemplo de imagem do <i>dataset</i> com as marcações dos quatro cantos da placa	43
Figura 15 – Diagrama do <i>pipeline</i> de classificação de caracteres isolados.	46
Figura 16 – Diagrama do <i>pipeline</i> de extração e classificação simultânea.	47
Figura 17 – Curvas de treinamento do modelo de detecção de placas	52
Figura 18 – Imagem em escala de cinza	54
Figura 19 – Extração da placa com base na bounding box	55
Figura 20 – Transformação de perspectiva com base nos quatro cantos da placa	56
Figura 21 – Medidas para remoção de bordas das placas do padrão antigo	57
Figura 22 – Medidas para remoção de bordas das placas do padrão Mercosul	58
Figura 23 – Remoção de bordas das placas	58
Figura 24 – Imagem de placa binarizada com o método de Sauvola	59
Figura 25 – Separação de caracteres para os dois modelos de placa	60
Figura 26 – Exemplo de limpeza de imagem de caractere com base em continuidade de objetos	61
Figura 27 – Diagrama da rede neural de classificação de caracteres	64
Figura 28 – Curvas de treinamento do modelo de classificação de números em placas do tipo Mercosul	65
Figura 29 – Curvas de treinamento do modelo de classificação de letras em placas do tipo Mercosul	65
Figura 30 – Curvas de treinamento do modelo de classificação de números em placas do tipo Brasil	66

Figura 31 – Curvas de treinamento do modelo de classificação de letras em placas do tipo Brasil	66
Figura 32 – Diagrama do bloco residual	68
Figura 33 – Diagrama da arquitetura completa da rede de extração e classificação simultânea	70
Figura 34 – Curvas de treinamento do modelo de classificação em placas do tipo Brasil	71
Figura 35 – Curvas de treinamento do modelo de classificação em placas do tipo Mercosul	72

LISTA DE TABELAS

Tabela 1 – Requisitos não funcionais de RF1	39
Tabela 2 – Requisitos não funcionais de RF2	39
Tabela 3 – Tamanhos do modelo de detecção de objetos YOLO26-pose	51
Tabela 4 – Sequência de operações do <i>backbone</i> e dimensões dos tensores	68
Tabela 5 – Acurácia por configuração de teste	77
Tabela 6 – Tempos médios de inferência por etapa (ms)	78
Tabela 7 – Comparativo geral entre configurações de teste	79

SUMÁRIO

1	INTRODUÇÃO	15
1.1	ESTRUTURA DO DOCUMENTO	16
2	FUNDAMENTAÇÃO TEÓRICA	18
2.1	MINERAÇÃO DE DADOS	18
2.1.1	CRISP-DM	18
2.2	REDES NEURAIS E <i>BACKPROPAGATION</i>	20
2.3	REDES NEURAIS CONVOLUCIONAIS	22
2.4	DETECÇÃO DE OBJETOS	24
2.4.1	YOLO	25
2.4.2	Detecção de pontos-chave	26
2.5	TÉCNICAS DE PROCESSAMENTO DE IMAGEM	27
2.5.1	Transformação de perspectiva	27
2.5.2	Limiarização adaptativa	28
2.5.3	Segmentação de caracteres	29
2.5.4	Filtragem de componentes conexos	29
2.6	<i>NEURAL PROCESSING UNIT</i>	30
2.7	QUANTIZAÇÃO DE REDES NEURAIS	31
3	DESCRIÇÃO DO PROBLEMA E REQUISITOS TÉCNICOS	33
3.1	DESCRIÇÃO DA EMPRESA	33
3.2	MOTIVAÇÃO	34
3.3	DESCRIÇÃO DO PROBLEMA	35
3.3.1	Criação do algoritmo de leitura automática de placas	35
3.3.2	Teste do algoritmo na placa Rock 3A	36
3.3.2.1	NPU da placa Rock 3A	37
3.4	REQUISITOS DA SOLUÇÃO	38
3.4.1	RF1: software de leitura automática de placas	38
3.4.1.1	Requisitos não funcionais	39
3.4.2	RF2: execução na placa Rock 3A	39
3.4.2.1	Requisitos não funcionais	39
4	SOLUÇÃO PROPOSTA	40
4.1	METODOLOGIA DE DESENVOLVIMENTO	40
4.2	DESCRIÇÃO DOS DADOS	41
4.2.1	Características das placas veiculares	41
4.2.2	Características do <i>dataset</i> utilizado	42
4.3	<i>PIPELINE</i> DE LEITURA DE PLACAS	43
4.3.1	Detecção e classificação da placa	44
4.3.2	Processamento da placa	44

4.3.3	Leitura da placa: caracteres isolados	45
4.3.4	Leitura da placa: extração e classificação simultâneas	46
4.4	EXECUÇÃO NA PLACA ROCK 3A	47
4.5	METODOLOGIA DE TESTE	47
5	DESENVOLVIMENTO E ANÁLISE DOS RESULTADOS OBTIDOS	49
5.1	PREPARAÇÃO DOS DADOS	49
5.1.1	Organização dos dados	50
5.2	MODELO DE DETECÇÃO DE PLACAS	50
5.2.1	Exportação do YOLO	52
5.3	ALGORITMO DE EXTRAÇÃO DE CARACTERES	52
5.3.1	Inferência com o YOLO	53
5.3.2	Criação de imagem da placa	54
5.3.3	Correção de perspectiva	55
5.3.4	Remoção das bordas e binarização	56
5.3.5	Separação dos caracteres	59
5.3.6	Limpeza dos caracteres	61
5.4	CRIAÇÃO DOS <i>DATASETS</i> DE CARACTERES E DE PLACAS	61
5.5	MODELOS DE CLASSIFICAÇÃO DE CARACTERES	62
5.5.1	Classificação de caracteres isolados	63
5.5.2	Extração e classificação simultânea	66
5.6	TRANSFORMAÇÃO DOS MODELOS PARA FORMATO COMPATÍVEL COM O <i>HARDWARE</i>	72
5.6.1	Resultados da conversão	73
5.6.2	Datasets de quantização	73
5.7	PREPARAÇÃO DO <i>HARDWARE</i>	73
5.7.1	Sistema operacional	74
5.7.2	Conexão de periféricos e <i>boot</i> do dispositivo	74
5.7.3	Configuração de SSH	74
5.7.4	Habilitação da NPU e instalação das bibliotecas	75
5.7.5	Download do código para o dispositivo	76
5.8	TESTES REALIZADOS	76
5.8.1	Resultados dos testes	77
6	CONCLUSÃO	80
6.1	SÍNTESE DO PROBLEMA, SOLUÇÃO E RESULTADOS	80
6.2	OBJETIVOS ATINGIDOS E LIMITAÇÕES	81
6.3	APRIMORAMENTOS POSSÍVEIS	81
6.4	IMPACTOS DO PROJETO	82
6.5	TRABALHOS FUTUROS	82
	REFERÊNCIAS	84

APÊNDICE A – DESCRIÇÃO 1	86
ANEXO A – DESCRIÇÃO 2	87

1 INTRODUÇÃO

A leitura automática de placas de veículos (do inglês *Automatic License Plate Recognition*, ALPR) é uma tecnologia de visão computacional com ampla aplicabilidade em controle de acesso, segurança e gestão de tráfego. Câmeras equipadas com essa capacidade já são comuns em estacionamentos, pedágios e condomínios, proporcionando automação e comodidade ao eliminar a necessidade de interação manual para liberação de acesso. Apesar de sua eficácia, os dispositivos comerciais disponíveis no mercado apresentam custo elevado, o que restringe sua adoção em aplicações de menor escala ou em cenários onde múltiplos dispositivos são necessários.

Este trabalho foi desenvolvido no contexto da Palmsoft Tecnologia, empresa de desenvolvimento de software sob demanda com atuação em sites, aplicativos, jogos e soluções de inteligência artificial. Um dos produtos da empresa é o *Aegis Access Control*, plataforma de controle de acesso para condomínios autônomos voltados ao aluguel de curto prazo. Nessa plataforma, hóspedes que realizam reservas em sites como *Booking* ou *AirBnB* recebem acesso automatizado ao condomínio por reconhecimento facial nas portas e por leitura de placa no estacionamento, sem necessidade de contato humano. A câmera com leitura automática de placas atualmente utilizada nessa solução é o modelo VIP 5460 LPR IA da Intelbras, cujo valor de mercado varia entre R\$ 5.700 e R\$ 7.500. O alto custo desse dispositivo motivou a Palmsoft a explorar alternativas mais acessíveis, capazes de viabilizar a tecnologia em um conjunto maior de aplicações. Paralelamente, a empresa demonstrou interesse em validar o uso de hardware de baixo custo para aplicações de inteligência artificial, tendo adquirido uma placa Radxa ROCK 3A, equipada com processador Rockchip RK3568 e uma NPU (*Neural Processing Unit*) de 0,8 TOPs.

O problema tratado neste trabalho é, portanto, o desenvolvimento de um sistema de leitura automática de placas de veículos que possa ser executado em hardware de baixo custo. Trata-se de um problema não trivial de visão computacional: as placas podem aparecer em diferentes posições e tamanhos na imagem, sujeitas a variações de iluminação, ângulo e perspectiva, e o espaço de placas possíveis é da ordem de centenas de milhões de combinações, inviabilizando abordagens de classificação direta. A importância de resolver esse problema é direta para a Palmsoft e seus clientes: uma solução funcional e econômica permitiria substituir câmeras de alto custo por dispositivos acessíveis, tornando o produto *Aegis Access Control* viável para um número maior de condomínios e ampliando o portfólio tecnológico da empresa em visão computacional embarcada.

O objetivo geral deste trabalho é desenvolver um software de leitura automática de placas de veículos e validar a viabilidade funcional de uma solução de baixo custo que execute esse software na placa Radxa ROCK 3A. Os objetivos específicos, que

definem os passos metodológicos do projeto, são: (1) preparar e organizar o conjunto de dados para treinamento e teste dos modelos; (2) treinar um modelo de detecção e classificação de placas baseado na família YOLO; (3) desenvolver um algoritmo de processamento de imagem para extração dos caracteres; (4) treinar modelos de classificação de caracteres; (5) converter os modelos para o formato RKNN, compatível com a NPU da ROCK 3A; e (6) avaliar a acurácia e o tempo de inferência do sistema completo, tanto no computador de desenvolvimento quanto na placa.

A solução proposta consiste em um *pipeline* de três etapas sequenciais. Na primeira, um modelo da família YOLO (especificamente o YOLO26n-pose) detecta a placa na imagem, identifica seu tipo (padrão antigo brasileiro ou padrão Mercosul) e localiza as coordenadas de seus quatro cantos. Na segunda etapa, um algoritmo de processamento de imagem converte a placa para escala de cinza, corrige sua perspectiva, remove bordas desnecessárias, binariza a imagem pelo método de Sauvola e segmenta os caracteres individualmente. Na terceira etapa, redes neurais convolucionais especializadas por tipo de caractere (letra ou número) e por modelo de placa classificam cada caractere extraído. Adicionalmente, foi desenvolvida uma abordagem alternativa em que um único modelo mais robusto realiza a extração e classificação de todos os caracteres diretamente da imagem da placa, sem necessidade da etapa de segmentação. O desenvolvimento seguiu o padrão CRISP-DM e utilizou o *dataset* público RodoSol-ALPR, composto por 10.000 imagens de veículos capturadas em um pedágio no Espírito Santo. Os modelos foram implementados em Python com as bibliotecas Ultralytics e PyTorch, exportados para o formato ONNX e, quando possível, convertidos para o formato RKNN com quantização *uint8* para execução na NPU.

Os resultados obtidos demonstraram a viabilidade técnica da solução. O *pipeline* de extração e classificação simultânea atingiu acurácia de 91,05%, enquanto o *pipeline* de caracteres isolados atingiu 85,63–85,75%. No computador de desenvolvimento (CPU), os tempos de inferência foram de 70,48 ms e 196,15 ms, respectivamente. Na ROCK 3A, o melhor desempenho foi obtido com o YOLO executado na NPU combinado ao modelo de extração e classificação simultânea na CPU, resultando em 464,10 ms com 91,05% de acurácia. A quantização dos modelos de classificação de caracteres não provocou perda perceptível de acurácia (variação inferior a 0,2 pontos percentuais), confirmando sua adequação para o ambiente embarcado. Esses resultados validam o uso da ROCK 3A como plataforma viável para uma câmera de leitura automática de placas de baixo custo, com potencial de redução de custo por unidade de uma ordem de grandeza em relação às câmeras comerciais.

1.1 ESTRUTURA DO DOCUMENTO

Este documento está organizado em seis capítulos. O Capítulo 2 apresenta a fundamentação teórica dos principais conceitos, técnicas e tecnologias utilizados no

projeto, incluindo redes neurais, redes neurais convolucionais, modelos de detecção de objetos, técnicas de processamento de imagem e quantização de redes neurais. O Capítulo 3 descreve o contexto empresarial, a motivação do projeto, a análise detalhada do problema e os requisitos funcionais e não funcionais da solução. O Capítulo 4 apresenta a solução proposta e a metodologia adotada, detalhando o *pipeline* de leitura de placas, as arquiteturas dos modelos desenvolvidos e a metodologia de teste. O Capítulo 5 discute o desenvolvimento efetivo da solução e os resultados obtidos nos testes realizados no computador e na placa ROCK 3A. Por fim, o Capítulo 6 apresenta as conclusões do trabalho, os objetivos atingidos, as limitações encontradas, os aprimoramentos possíveis e as sugestões de trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo é discutida a fundamentação teórica dos principais conceitos, técnicas e tecnologias utilizados no projeto, incluindo:

- Treinamento de modelos de aprendizado de máquina
- Redes neurais e *backpropagation*
- Redes neurais convolucionais
- Modelos de detecção de objetos
- Técnicas de transformação de imagens para visão computacional
- *Neural Processing Unit*
- Quantização de redes neurais

Essa fundamentação forma a base necessária para a criação, o desenvolvimento e a validação do algoritmo de leitura de placas de carro desenvolvido neste projeto, além de sua execução no *hardware* específico disponível.

2.1 MINERAÇÃO DE DADOS

O processo padrão de mineração de dados consiste no ato de utilizar algoritmos para extrair informações relevantes de um banco de dados (Fayyad; Piatetsky-Shapiro; Smyth, 1996). Este processo já possui uma forma bem definida e aceita, conhecida como CRISP-DM (*Cross-Industry Standard Process for Data Mining*) (Chapman et al., 2000).

O treinamento de modelos de aprendizado de máquina se enquadra como um processo de mineração de dados, pois nada mais é do que encontrar um modelo que, de alguma forma, represente o relacionamento entre diferentes aspectos do conjunto de dados. O modelo, nesse caso, representa a informação que se deseja extrair dos dados.

Um bom modelo descreverá com precisão o relacionamento entre os aspectos desejados do conjunto de dados e será, portanto, útil para extrair novas informações de novos dados.

2.1.1 CRISP-DM

O padrão CRISP-DM é uma sequência de etapas a partir das quais é possível extrair informações úteis de um banco de dados, sendo que cada etapa é definida por tarefas e um resultado esperado.

As etapas são:

1. Entendimento do problema

A primeira fase consiste em entender as características do problema e os requisitos da solução, criando assim uma definição de problema de mineração de dados e um plano preliminar para atingir os objetivos (Chapman et al., 2000).

2. Entendimento dos dados

A fase seguinte envolve familiarizar-se com os dados à disposição. Isso envolve descobrir *insights* iniciais nos dados, detectar problemas de qualidade e identificar subconjuntos interessantes (Chapman et al., 2000).

3. Preparação dos dados

Essa etapa consiste nas atividades necessárias para construir o *dataset* final a partir dos dados brutos. Pode envolver diversas tarefas, como limpeza dos dados, seleção de atributos relevantes e transformação dos dados (Chapman et al., 2000).

4. Modelagem

A modelagem envolve selecionar, aplicar e calibrar diversas técnicas de modelagem, buscando encontrar um modelo ótimo para o problema que se deseja resolver (Chapman et al., 2000).

5. Avaliação

Após chegar a um ou mais modelos que aparentam ser apropriados, uma avaliação final deve ser feita para concluir se atendem aos requisitos de negócio. Ao final dessa avaliação deve-se chegar a uma conclusão relativa ao uso ou não de cada modelo para o objetivo final (Chapman et al., 2000).

6. Deployment

Por fim, a última etapa é o momento em que o conhecimento obtido é preparado para uso pelo cliente do projeto. Essa preparação depende muito dos requisitos, podendo ser na forma de um relatório ou de um sistema automatizado de mineração de dados (Chapman et al., 2000).

A figura 1 mostra um diagrama do processo e demonstra como algumas das etapas possuem uma relação de troca, de forma que o conhecimento ou os desafios encontrados em uma dada etapa podem fazer com que seja necessário voltar para uma etapa anterior.

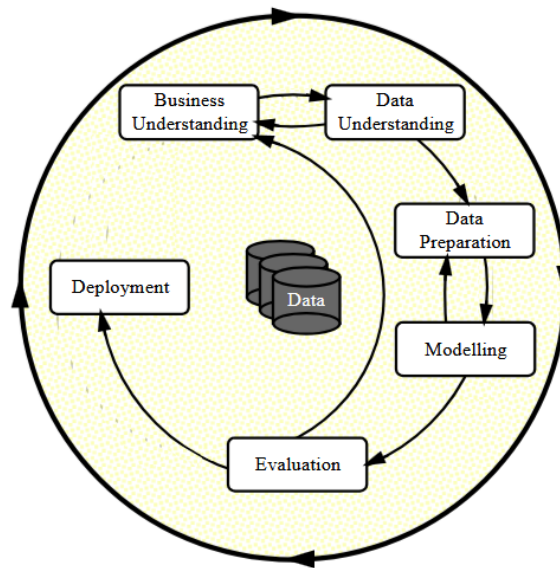


Figura 1 – Fases do processo CRISP-DM (Chapman et al., 2000)

2.2 REDES NEURAIS E *BACKPROPAGATION*

As redes neurais artificiais e o algoritmo de *backpropagation* são alguns dos fundamentos mais importantes da inteligência artificial moderna. As redes neurais permitem a criação de modelos capazes de aproximar relações não lineares e complexas entre dados, sendo que o limite da complexidade dessas relações se dá apenas pela limitação computacional ou pelo volume de dados utilizados no ajuste do modelo. Já o algoritmo de *backpropagation* permite ajustar o modelo de redes neurais aos dados, minimizando uma função de erro.

Esses conceitos são utilizados para criar modelos de inteligência artificial voltados para visão computacional, processamento de linguagem natural, séries temporais, IA generativa e inúmeras outras aplicações.

O tipo clássico de redes neurais é composto por uma série de camadas de processamento. Cada camada processa a saída da camada anterior em sequência até chegar no *output* final. A saída $a_{(c)}$ de uma camada c é definida por:

$$a_{(c)} = f(z_{(c)}), \quad (1)$$

sendo que f é a função de ativação da camada e $z_{(c)}$ é definido como:

$$z_{(c)} = W_{(c)} \times a_{(c-1)} + b_{(c)}, \quad (2)$$

onde:

- $W_{(c)}$ é a matriz de pesos da camada,
- $a_{(c-1)}$ é a saída da camada anterior,

- $b_{(c)}$ é o vetor de viés da camada.

A função de ativação f é responsável por conferir não linearidade à camada, pois sem ela a camada não passaria de uma transformação linear, capaz apenas de representar relações lineares entre os dados, limitando muito sua aplicabilidade.

Existem diversas funções de ativação que podem ser usadas, mas as mais comuns são:

- **Sigmoide:** $\sigma(x) = \frac{1}{1+e^{-x}}$
- **Tangente hiperbólica:** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **ReLU (*Rectified Linear Unit*):** $\text{ReLU}(x) = \begin{cases} x ; & x > 0 \\ 0 ; & x \leq 0 \end{cases}$

A matriz de pesos $W_{(c)}$ e o vetor de viés $b_{(c)}$ são compostos por valores ajustáveis que devem ser modificados até que o modelo esteja satisfatório.

Para que os pesos do modelo sejam ajustados, é utilizado o algoritmo de *back-propagation* para calcular o gradiente da função de erro em relação aos pesos da rede. A função de erro varia conforme o objetivo daquela rede neural e a função de ativação da última camada.

Para calcular o gradiente, é necessário calcular o delta $\delta_{(c)}$ de cada camada.

Para a camada de saída da rede (C), o delta é dado por:

$$\delta_{(C)} = \hat{y} - y, \quad (3)$$

sendo que $\hat{y}$ é o valor predito e y é o valor esperado (verdadeiro).

Para as demais camadas, o delta é definido como:

$$\delta_{(c)} = W_{(c+1)}^T \times \delta_{c+1} \odot f'(z_{(c)}). \quad (4)$$

Por fim, o gradiente da função de erro L em relação aos pesos fica:

$$\frac{\partial L}{\partial W_{(c)}} = \delta_{(c)} \times a_{(c-1)}^T, \quad (5)$$

$$\frac{\partial L}{\partial b_{(c)}} = \delta_{(c)}. \quad (6)$$

Nota-se que, enquanto a saída de uma camada é calculada com base na saída da camada anterior, o gradiente de uma camada é calculado com base no delta da camada seguinte. Essa propagação retroativa dos erros é o motivo de o algoritmo de cálculo do gradiente ser chamado de *backpropagation* (Rumelhart; Hinton, Geoffrey E.; Williams, 1986).

O último fator importante a ser discutido no que diz respeito às redes neurais artificiais é o algoritmo de atualização dos pesos com base no gradiente. Existem

diversos algoritmos diferentes que podem ser utilizados, porém o mais clássico e simples é o SGD (*Stochastic Gradient Descent*), no qual a um peso $v_{(c)}$ é atribuído:

$$v_{(c)} \leftarrow v_{(c)} - \eta \frac{\partial L}{\partial v_{(c)}}, \quad (7)$$

onde η é a taxa de aprendizagem.

2.3 REDES NEURAIIS CONVOLUCIONAIS

A eficácia de redes neurais convolucionais (CNNs) para tarefas de classificação e segmentação de imagens já é comprovada (Krizhevsky; Sutskever; Hinton, Geoffrey E, 2012), sendo uma solução amplamente utilizada em diversas aplicações.

Essa modalidade de rede neural tem como característica as camadas convolucionais, que apresentam vantagens em relação às camadas *fully connected* tradicionais para tarefas de interpretação de imagens ou mesmo de sinais (LeCun et al., 1998).

A limitação das camadas *fully connected* para esse tipo de tarefa se dá pelo fato de que esse tipo de camada produz sua saída com base no relacionamento entre todos os *inputs* da camada uns com os outros. Isso é ótimo para encontrar relações entre diferentes atributos quando se trabalha com dados tabulares, por exemplo, mas não é eficaz para identificar características específicas que podem estar localizadas em diferentes partes de uma imagem.

Para esse tipo de tarefa, as camadas convolucionais são mais apropriadas, pois utilizam conectividade local e compartilhamento de pesos, reduzindo drasticamente a quantidade de parâmetros necessários para se obter um resultado satisfatório.

A camada convolucional funciona deslizando pelo *input* um filtro chamado *kernel*, que nada mais é do que uma matriz de pesos de dimensão pequena, como 3×3 ou 5×5 . Em cada posição do *kernel* é então feita uma transformação linear de um conjunto local de valores utilizando os pesos do *kernel*.

Fazendo isso em todo o *input*, obtém-se um mapa de características, que destaca padrões identificados em cada parte do *input*. Esse mapa de características pode então ser alimentado para outra camada convolucional e assim por diante. Além disso, para cada *input* pode-se (e deve-se) ter múltiplos *kernels*, de forma a gerar múltiplos mapas de características em cada camada, cada um destacando aspectos distintos do *input*. A figura 2 exemplifica o funcionamento da camada convolucional.

A equação da convolução 2D discreta utilizada nas camadas convolucionais para imagens é:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j), \quad (8)$$

sendo que I é a imagem e K é o *kernel*.

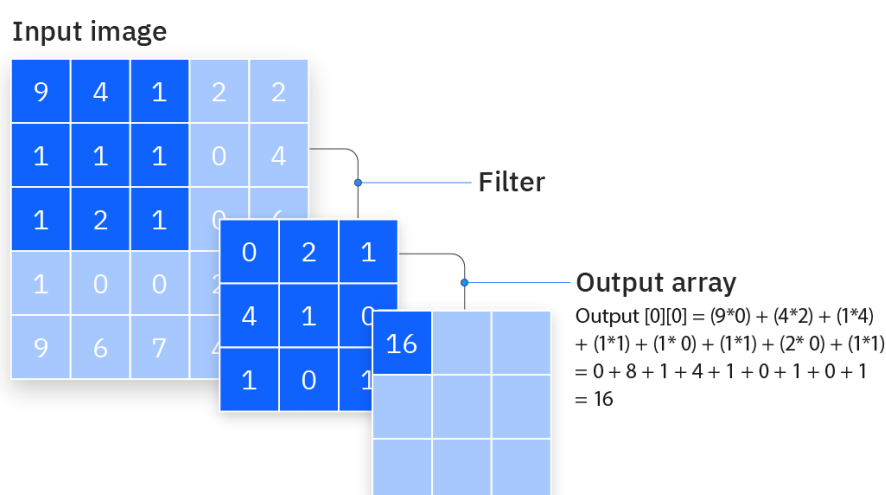


Figura 2 – Camada convolucional (IBM Inc., 2024)

Redes neurais convolucionais também costumam utilizar camadas de *pooling* após cada camada convolucional. Essa camada utiliza um método de subamostragem para reduzir a dimensão de seu *input*. Suas duas formas mais comuns são o *max pooling* e o *average pooling*.

A ideia do *pooling* é dividir o *input* em janelas não sobrepostas e transformar cada janela em um único valor escalar, aplicando sobre ela o cálculo da média ou extraindo seu valor máximo, a depender da técnica utilizada.

As camadas convolucionais têm a tarefa de extrair atributos da imagem e são eficazes em identificar atributos específicos independentemente da região da imagem em que estejam localizados. Após a extração dos atributos, porém, ainda é necessário utilizá-los de alguma forma. Em tarefas de classificação de imagens é muito comum alimentar os atributos extraídos para camadas *fully connected*, que aprendem padrões dos atributos e dos relacionamentos entre eles e utilizam isso para classificar a imagem.

A forma padrão de um modelo de redes neurais convolucionais para classificação de imagens, portanto, é uma sequência de camadas convolucionais e camadas de *pooling*, seguidas por uma sequência de camadas *fully connected* que culminam no *output* do modelo. A figura 3 mostra um diagrama que exemplifica a arquitetura de uma rede neural convolucional para classificação de imagens.

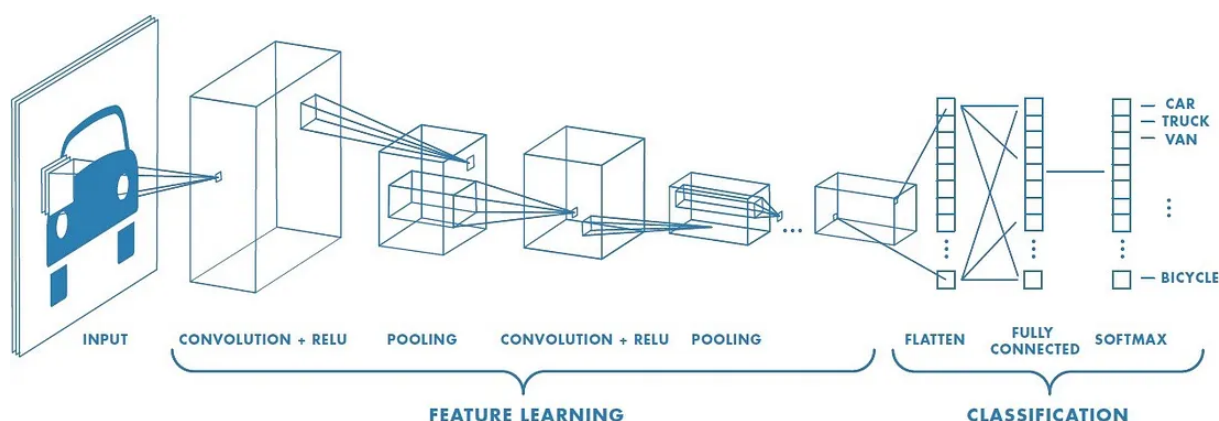


Figura 3 – Rede neural convolucional para classificação de imagens (Saha, 2018)

2.4 DETECÇÃO DE OBJETOS

A tarefa de detecção de objetos é um campo desafiador dentro da área mais ampla de visão computacional. Essa tarefa consiste em identificar objetos específicos dentro de uma imagem, incluindo sua classe e localização. É, portanto, uma tarefa mais complexa que a classificação de imagens, que se preocupa simplesmente em atribuir uma classe à imagem toda, enquanto a detecção de objetos deve extrair da imagem regiões retangulares chamadas *bounding boxes* e atribuir uma classe a cada uma dessas regiões.

Isso permite diversas aplicações, como:

- Rastrear a movimentação de um objeto ao longo de um vídeo;
- Verificar quantas instâncias de um dado objeto existem na imagem;
- Extrair da imagem original uma nova imagem contendo apenas o objeto de interesse.

A figura 4 exemplifica a tarefa de detecção de objetos em visão computacional.

Os modelos mais comuns de detecção de objetos costumam ser divididos em duas categorias: detectores de duas etapas e detectores de uma etapa (Zou et al., 2019). Os modelos de duas etapas primeiro dividem a imagem em regiões candidatas, onde a probabilidade de haver objetos é alta, e posteriormente geram as *bounding boxes* e classificam as regiões. Modelos de duas etapas costumam ser mais precisos, mas o tempo de inferência é mais alto.

Modelos de uma etapa, por outro lado, analisam a imagem inteira e tentam classificar as regiões e gerar as *bounding boxes* diretamente. Esses modelos têm a vantagem de serem mais rápidos, portanto mais apropriados para aplicações de tempo real.

Como tarefas de detecção de objetos são muito utilizadas em aplicações de tempo real, a velocidade de inferência é uma métrica muito importante na avaliação

desse tipo de modelo, além da precisão do resultado. Nessa linha, modelos que colocam grande ênfase na velocidade de inferência ganharam grande tração no campo da detecção de objetos. Um desses modelos (ou família de modelos) é o YOLO, discutido a seguir.

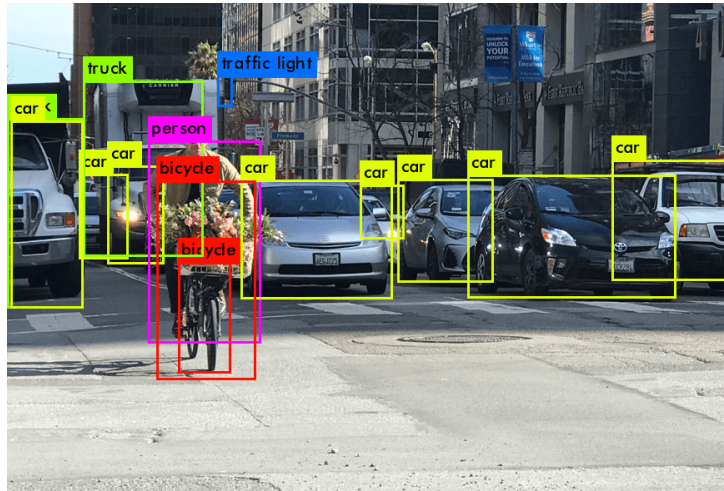


Figura 4 – Exemplo de detecção de objetos

2.4.1 YOLO

Os modelos da família YOLO (*You Only Look Once*) são modelos muito rápidos de detecção em uma etapa, portanto apropriados para aplicações de tempo real. Quando os modelos YOLO surgiram, apenas modelos de duas etapas eram utilizados para detecção de objetos com redes neurais, mas a inovação proposta com os modelos de uma etapa se mostrou extremamente eficiente e se tornou amplamente utilizada (Redmon et al., 2016).

Modelos YOLO tratam a tarefa de detecção de objetos como uma única tarefa de regressão, diferente dos modelos de duas etapas, que se baseiam em um *pipeline* mais complexo. O modelo funciona dividindo a imagem em um grid $S \times S$, e cada célula do grid é responsável por detectar os objetos cujo centro está dentro dela. Cada célula prevê B *bounding boxes*, sendo que cada uma é representada por 5 valores: as coordenadas (x, y) do centro relativo à célula, a altura, a largura e a confiança. A confiança representa a *IoU* (*Intersection over Union*) da *bounding box* prevista com a verdadeira, caso haja um objeto na célula. Se não houver objeto, a confiança deve ser igual a zero (Redmon et al., 2016):

$$IoU = \frac{\text{Área de Interseção}}{\text{Área de União}}. \quad (9)$$

Cada célula também produz C probabilidades de classe, cada uma representando a probabilidade de aquela célula conter um objeto de uma dada classe.

A figura 5 mostra uma representação do funcionamento da rede YOLO.

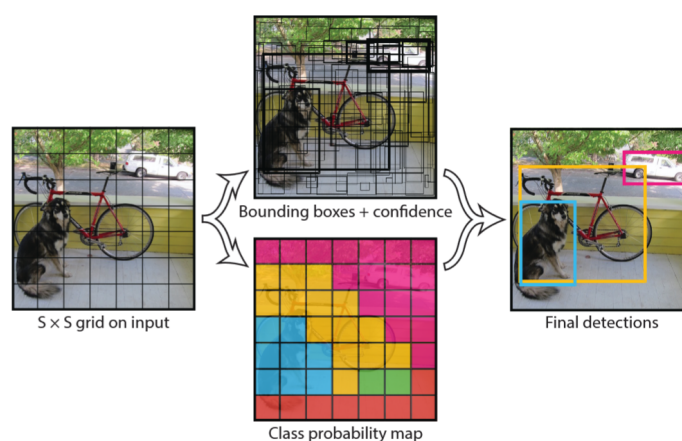


Figura 5 – Diagrama de funcionamento do modelo YOLO (Redmon et al., 2016)

2.4.2 Detecção de pontos-chave

Dentro da área da detecção de objetos, há também uma subdivisão chamada detecção de pontos-chave. Essa tarefa consiste em, além de detectar um objeto, detectar também a posição de uma série de pontos importantes do objeto. Esses pontos podem ser relevantes em vários contextos, pois oferecem informação adicional a respeito da posição ou do formato do objeto.

Uma aplicação muito comum da detecção de pontos-chave é estimar a posição de seres humanos. Ao identificar a posição de diversas partes do corpo, como ombros, cotovelos e mãos, relativas umas às outras, é possível inferir a postura da pessoa e se ela está realizando certas ações, como andar ou sentar.

A figura 6 mostra um exemplo de detecção de pontos-chave no corpo humano.

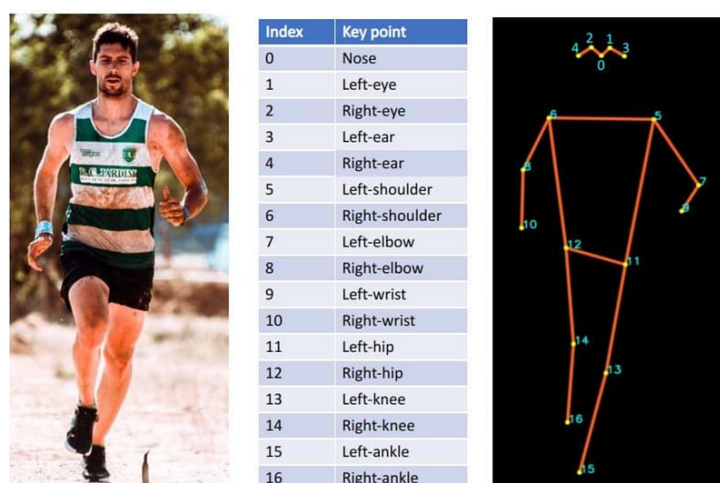


Figura 6 – Exemplo de detecção de pontos-chave em um corpo humano

2.5 TÉCNICAS DE PROCESSAMENTO DE IMAGEM

Inúmeros algoritmos de manipulação de imagens são utilizados em visão computacional desde muito antes do surgimento das redes neurais. Alguns desses algoritmos ainda são extremamente relevantes, especialmente no tratamento de imagens que possuem padrões conhecidos.

Essas técnicas são úteis em especial para extrair características das imagens e prepará-las para serem alimentadas por modelos de inteligência artificial, removendo ruídos e destacando as características mais relevantes da imagem.

As principais técnicas utilizadas neste projeto foram:

- Transformação de perspectiva
- Limiarização adaptativa
- Segmentação de caracteres
- Filtragem de componentes conexos

2.5.1 Transformação de perspectiva

A transformação de perspectiva é uma operação utilizada para corrigir a perspectiva de uma figura 2D dentro de uma imagem. Ela permite que objetos inclinados ou rotacionados sejam alinhados com base nas coordenadas dos quatro cantos do objeto. Esse tipo de transformação não preserva paralelismo, comprimento e ângulo, mas preserva colinearidade e incidência (TheAILearner, 2023).

A transformação de perspectiva pode ser expressa como:

$$\begin{bmatrix} t_i x' \\ t_i y' \\ t_i \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & b_1 \\ a_3 & a_4 & b_2 \\ c_1 & c_2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad (10)$$

sendo que x' e y' são os pontos transformados, ou seja, as coordenadas (x, y) dos pontos na imagem de saída, enquanto x e y são os pontos originais. O termo t_i é um fator de escala.

A matriz de transformação

$$M = \begin{bmatrix} a_1 & a_2 & b_1 \\ a_3 & a_4 & b_2 \\ c_1 & c_2 & 1 \end{bmatrix} \quad (11)$$

é formada por três componentes: a matriz $\begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix}$, responsável por definir transformações como rotação e escala; o vetor $\begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$, responsável por definir a translação; e o vetor de projeção $\begin{bmatrix} c_1 & c_2 \end{bmatrix}$.

Como a matriz de transformação possui 8 incógnitas, são necessárias 8 equações para que seja resolvida. Para isso, são selecionados quatro pontos (x, y) na imagem original e quatro pontos (x', y') na imagem de saída. Dessa forma é possível obter a matriz de transformação capaz de mapear todos os pontos da imagem original para a imagem final.

A figura 7 exemplifica o que ocorre com uma imagem na transformação de perspectiva.

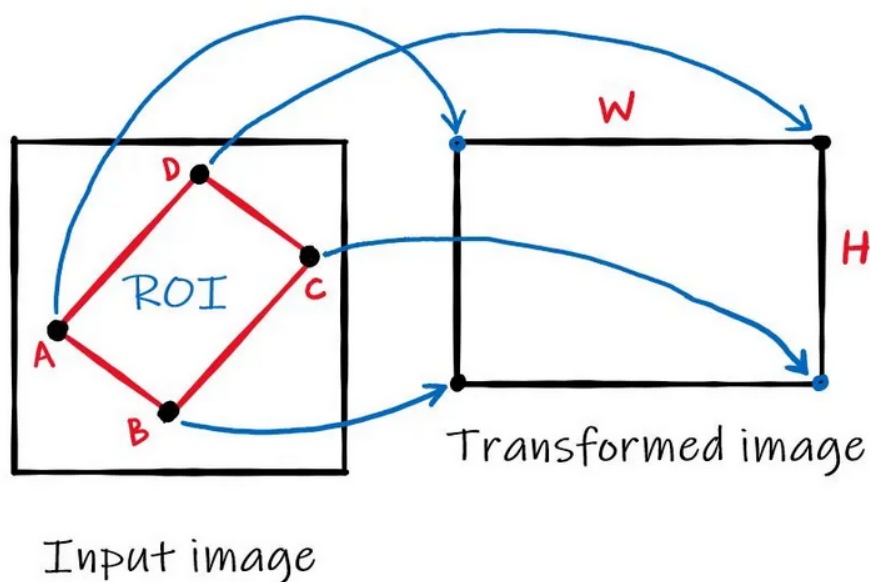


Figura 7 – Transformação de perspectiva (TheAILearner, 2023)

2.5.2 Limiarização adaptativa

A limiarização é uma técnica muito utilizada em visão computacional para criar máscaras a partir de imagens. Ela consiste em definir um limiar e classificar todos os pixels da imagem em 0 (preto) ou 1 (branco), dependendo se o valor original do pixel fica acima ou abaixo do limiar definido. Uma aplicação muito comum desse conceito em visão computacional é a separação de um objeto do fundo da imagem.

Independentemente da aplicação, a escolha do valor do limiar impacta profundamente a imagem binarizada resultante. Uma escolha inadequada pode fazer com que partes significativas do objeto sejam apagadas ou que ruídos do fundo se misturem ao objeto. Portanto, a aplicação de uma boa técnica para selecionar o limiar se faz necessária.

Outro fator relevante a considerar é que, em muitos casos, a iluminação faz com que certas partes da imagem sejam mais escuras que outras. Isso torna interessante utilizar um limiar diferente em diferentes partes da imagem, de forma a evitar distorções causadas por variações de iluminação.

Um método para determinação desse limiar variável é o método de Sauvola (Sauvola; Pietikäinen, 2000), que determina um limiar ideal para cada pixel da imagem usando os pixels vizinhos como base.

O método funciona calculando a média e o desvio padrão local para cada pixel com base em uma janela. A média local para um dado pixel de coordenadas (x, y) na imagem I é definida como:

$$\mu(x, y) = \frac{1}{W^2} \sum_{i=-\frac{W}{2}}^{\frac{W}{2}} \sum_{j=-\frac{W}{2}}^{\frac{W}{2}} I(x + i, y + j), \quad (12)$$

sendo que W é o tamanho da janela escolhido.

Já o desvio padrão local desse mesmo pixel é definido como:

$$\sigma(x, y) = \sqrt{E[I^2](x, y) - \mu(x, y)^2}, \quad (13)$$

sendo que $E[I^2](x, y)$ é a média local do pixel de coordenadas (x, y) na imagem I^2 .

Por fim, o limiar de Sauvola do pixel de coordenadas (x, y) na imagem I é:

$$T(x, y) = \mu(x, y) \cdot \left[1 + k \left(\frac{\sigma(x, y)}{R} - 1 \right) \right], \quad (14)$$

sendo que R é o valor máximo teórico de σ e k é um parâmetro de sensibilidade, tipicamente 0,3.

2.5.3 Segmentação de caracteres

A segmentação de caracteres serve para dividir uma imagem contendo múltiplos caracteres em um conjunto de imagens que contém apenas um caractere em cada. Partindo do pressuposto de que a imagem original é binarizada, contendo portanto apenas pixels com valores zero ou um, a segmentação de caracteres pode ser feita a partir da detecção de espaços em branco entre os caracteres (Casey; Lecolinet, 1996).

2.5.4 Filtragem de componentes conexos

A filtragem de componentes conexos é um processo que pode ser usado para isolar o objeto de interesse em uma imagem binarizada e remover ruído.

Ao binarizar uma imagem, é comum que pixels que não pertencem ao objeto analisado sejam classificados como parte do objeto, o que pode atrapalhar a análise

da imagem. Às vezes esses pixels estão conectados ao objeto, o que torna mais difícil identificá-los, mas às vezes estão desconectados, o que facilita sua identificação. Quando os pixels de ruído estão desconectados do objeto, é possível identificá-los ao separar a imagem em *blobs*, regiões isoladas umas das outras.

Cada *blob* representa uma região da imagem na qual todos os pixels possuem valor 1 e têm conectividade com pelo menos um outro pixel que também pertence àquele *blob*. A figura 8 exemplifica a separação de regiões da imagem em *blobs* conexos, em que cada cor representa um *blob* diferente.



Figura 8 – Exemplo de separação de regiões por componentes conexos

A imagem separada em *blobs* pode ser representada como uma matriz na qual a cada posição é atribuído um número a depender do *blob* ao qual aquela posição pertence. Esses números são os rótulos das regiões.

Para realizar a filtragem, basta determinar qual é o *blob* com maior área e eliminar os demais. A área de um *blob*, nesse caso, nada mais é do que a quantidade de pixels pertencentes a ele.

2.6 NEURAL PROCESSING UNIT

A NPU (*Neural Processing Unit*) é um processador especializado em redes neurais, otimizado para operações matriciais e convolucionais, que são algumas das operações mais comuns nesse tipo de modelo.

O intuito da NPU é permitir a execução de modelos de redes neurais de forma eficiente em aplicações embarcadas, que geralmente não são equipadas com CPUs ou GPUs poderosas. Para isso, a NPU sacrifica flexibilidade em prol da performance em uma tarefa específica.

A arquitetura de uma NPU consiste basicamente em um grid de *processing engines* (PEs), sendo que cada uma realiza a computação de um neurônio (multiplicação, acumulação e ativação). A figura 9 mostra a arquitetura da NPU (Chen et al., 2020).

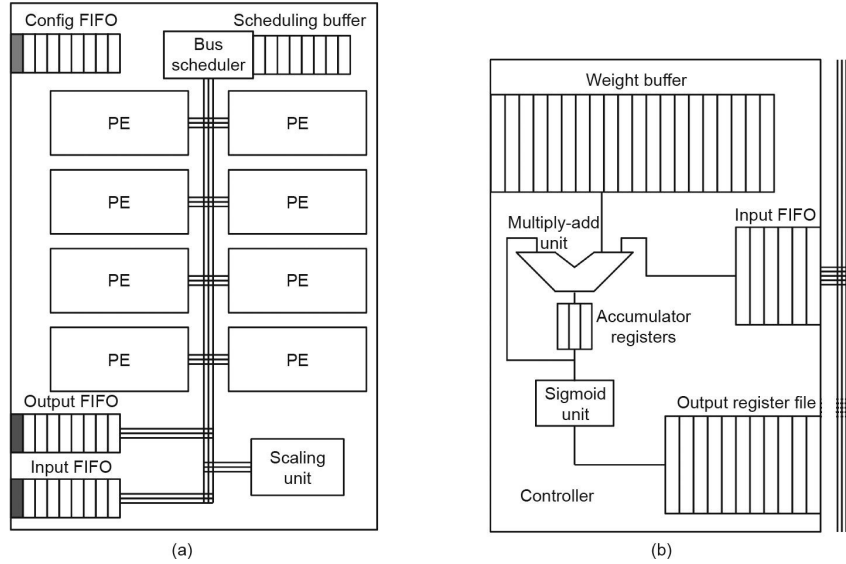


Figura 9 – A NPU. (a) NPU de 8 *processing engines*; (b) Fila de uma *processing engine* (Chen et al., 2020)

2.7 QUANTIZAÇÃO DE REDES NEURAIS

No contexto de redes neurais artificiais, a quantização é o processo de converter os pesos e as ativações da rede, que originalmente são valores reais (normalmente de 32 *bits*), para valores inteiros, normalmente de 8 *bits*. Esse processo reduz a precisão numérica, mas pode gerar ganhos significativos de performance, além de reduzir o espaço em memória ocupado pelo modelo (Jacob et al., 2018).

Em um possível esquema de quantização proposto por (Jacob et al., 2018), a relação entre um valor quantizado q e um valor real r é definida como:

$$r = S(q - Z), \quad (15)$$

onde S e Z são os parâmetros de quantização, diferentes para cada vetor de pesos e ativações da rede. O parâmetro Z (*zero-point*) representa o valor de q quando $r = 0$, permitindo que o erro da quantização seja nulo para $r = 0$. Já a constante S (*scale*) é um valor real positivo.

É possível realizar a quantização após o treinamento do modelo, mas em muitos casos é recomendado que o modelo seja treinado já aplicando uma forma de quantização simulada, para que aprenda a compensar os erros de quantização.

Para se obter o valor quantizado q dado um valor real r , é aplicado o seguinte algoritmo:

$$\text{clamp}(r; a, b) := \min(\max(r, a), b), \quad (16)$$

$$s(a, b, n) := \frac{b - a}{n - 1}, \quad (17)$$

$$q(r; a, b, n) := \left\lfloor \frac{\text{clamp}(r; a, b) - a}{s(a, b, n)} \right\rfloor s(a, b, n) + a, \quad (18)$$

onde:

- r é o valor real sendo quantizado,
- $[a; b]$ é o intervalo de quantização, os valores mínimo e máximo esperados para r ,
- n é o número de níveis de quantização (256 para 8 *bits*),
- $\lfloor . \rfloor$ representa arredondamento para o inteiro mais próximo.

Quantizando desta forma, é possível obter um bom balanço entre performance e acurácia do modelo (Jacob et al., 2018).

3 DESCRIÇÃO DO PROBLEMA E REQUISITOS TÉCNICOS

Neste capítulo será discutido de forma mais aprofundada o problema que este projeto se propõe a resolver. Isso envolve, além de uma observação detalhada do problema e seus requisitos, o contexto no qual o problema está inserido dentro da empresa e as motivações para resolvê-lo.

3.1 DESCRIÇÃO DA EMPRESA

A Palmsoft Tecnologia é uma empresa de desenvolvimento de software sob demanda. Produz sites, aplicativos, jogos, soluções de inteligência artificial e outros produtos de software.

Um dos produtos desenvolvidos pela Palmsoft é o *Aegis Access Control*, uma plataforma que gerencia controle de acesso em condomínios equipados com fechaduras eletrônicas de desbloqueio por reconhecimento facial.

O objetivo do *Aegis Access Control* é atender proprietários e gestores de apartamentos que fazem aluguel de curto prazo, permitindo que automatizem o processo de concessão de acesso aos imóveis. Quando o hóspede faz uma reserva, o gestor do apartamento acessa uma interface *web* onde cadastra o hóspede e as informações da reserva, como data de início e de fim. O hóspede então recebe um *e-mail* com um *link* que o leva a uma página onde deve cadastrar uma foto do seu rosto e outras informações complementares, como a placa do carro. Com base nas informações da reserva, o sistema libera o acesso do hóspede ao condomínio e ao apartamento específico que foi alugado, apenas durante o período da estadia. Em cada porta do condomínio há um controlador de acesso facial, que destrava a fechadura com base no rosto do usuário. O cadastro do rosto nos controladores de acesso é feito utilizando a foto enviada pelo usuário.

Para completar a experiência, o estacionamento é equipado com uma câmera com leitura automática de placas integrada, conectada ao portão. Da mesma forma que as imagens faciais são enviadas pelo sistema aos controladores de acesso das portas, as placas dos carros dos hóspedes também são enviadas às câmeras. Quando o carro do hóspede se aproxima, a câmera lê a placa do veículo, identifica que aquela placa possui acesso ao condomínio e envia um sinal ao portão para que ele abra.

Dessa forma, o hóspede recebe uma experiência completamente automatizada: chega ao condomínio e seu acesso já está liberado sem que precise ter contato com nenhum atendente.

Além da experiência do hóspede, essa tecnologia é muito benéfica para os gestores dos apartamentos, que conseguem administrá-los de forma remota. Quando precisam conceder acesso a um prestador de serviço que irá limpar o apartamento, por exemplo, isso também é feito pelo mesmo processo.

3.2 MOTIVAÇÃO

Dentro do contexto do condomínio autônomo, as câmeras instaladas na garagem são um fator importante, que adiciona conforto à experiência do cliente. Essas câmeras são equipadas com tecnologia de visão computacional capaz de ler a placa de um veículo em questão de milissegundos, com ótima precisão.

Câmeras como essa já fazem parte do dia a dia de muitas pessoas, em diversas aplicações. Uma delas, muito comum, é o controle de acesso em estacionamentos pagos, como em shoppings ou supermercados. Elas permitem que o sistema registre a placa do carro de cada cliente e a associe ao seu *ticket* de estacionamento. Uma das vantagens disso é que, caso o cliente perca o *ticket* impresso, o sistema pode gerar um novo sem problemas, mantendo o mesmo horário de entrada. Outra vantagem é que, quando o cliente se aproxima da cancela de saída, a placa do seu carro é rapidamente detectada e o sistema pode verificar se aquele cliente pagou o estacionamento sem que ele precise apresentar o *ticket* impresso. Isso evita congestionamentos na saída e melhora o fluxo de veículos. Esse sistema também pode ser utilizado para conceder acesso gratuito a funcionários, permitindo a entrada deles sem retirada de *ticket*, com base na placa do carro.

Outra aplicação para esse tipo de câmera é em sistemas de segurança de estacionamentos. Ao detectar placas de carros, o sistema pode gerar *logs* de entrada e saída de veículos. Isso auxilia muito na auditoria dos vídeos das câmeras de segurança, pois os momentos mais relevantes ficam registrados. Dessa maneira, fica fácil responder perguntas como: “Em qual data e hora o carro com a placa X acessou o estacionamento?”. Além disso, é possível identificar facilmente o acesso de veículos não autorizados a um estacionamento privado.

No condomínio que utiliza a solução do *Aegis Access Control*, a câmera utilizada é um dispositivo da Intelbras, modelo VIP 5460 LPR IA, cujo valor de mercado varia entre R\$ 5.700 e R\$ 7.500. O alto valor desse dispositivo motivou a exploração dessa tecnologia para o desenvolvimento de alternativas mais acessíveis, que possam ser utilizadas em aplicações onde o custo é um fator determinante.

Outro fator motivador é que a Palmsoft Tecnologia demonstrou interesse em testar a performance de *hardware* de baixo custo para aplicações de inteligência artificial, com o objetivo de validar o equipamento para o desenvolvimento de produtos nessa área. Para isso, foi adquirida uma placa Rock 3A, da Radxa, equipada com uma NPU de 0,8 TOPs.

Portanto, unindo a motivação do desenvolvimento de uma alternativa de baixo custo para câmera com leitura automática de placas de carro à de validar uma placa acessível para aplicações de inteligência artificial, foi concebido um projeto que desenvolvesse um modelo de leitura de placas de carro e validasse sua execução na placa Rock 3A.

O objetivo deste projeto, portanto, não é criar um protótipo funcional, mas desenvolver e validar um algoritmo de leitura de placas de carro que recebe uma imagem e retorna os caracteres da placa do veículo contido nela (se houver), e analisar a performance desse algoritmo ao ser executado na placa Rock 3A, validando assim a viabilidade de criação de um produto que utilize o algoritmo desenvolvido nesse *hardware*.

3.3 DESCRIÇÃO DO PROBLEMA

O objetivo geral deste projeto pode ser definido como:

“Desenvolver um software de leitura automática de placas de veículos e validar a viabilidade funcional de uma solução de baixo custo que execute esse software em uma placa Rock 3A.”

O problema, portanto, pode ser dividido em duas partes:

- Desenvolver o *software* de leitura automática de placas de veículos;
- Validar a placa Rock 3A como um *hardware* viável para execução desse *software*.

A primeira parte envolve o desenvolvimento de um algoritmo complexo de visão computacional capaz de ler os caracteres da placa de um carro ao longo de múltiplas etapas, exigindo conhecimento técnico acerca dos diferentes modelos de inteligência artificial para processamento de imagens e das técnicas de visão computacional.

Já a segunda parte envolve executar e avaliar a performance do algoritmo no *hardware*. Para isso, é necessária compreensão do dispositivo a ser utilizado, dos requisitos para executar modelos de redes neurais na NPU e das configurações necessárias.

3.3.1 Criação do algoritmo de leitura automática de placas

O *software* de leitura automática de placas de veículos a ser desenvolvido é um algoritmo que recebe uma imagem como entrada e devolve uma sequência de caracteres correspondente à placa do carro contido na imagem (se houver). Esse algoritmo deve poder ser executado na placa Rock 3A, portanto devem ser observadas as limitações desse *hardware*, que serão discutidas posteriormente.

As características desejáveis do algoritmo são principalmente:

- **Acurácia:** O algoritmo deve inferir corretamente a maioria das placas dos carros contidos nas imagens que lhe são entregues. Um leitor automático de placas que comete muitos erros não tem utilidade, portanto a proporção de placas inferidas

corretamente em relação ao total de imagens analisadas deve ser a mais alta possível.

- **Velocidade:** Para muitas aplicações, a velocidade é crítica, especialmente quando se deseja inferir a placa de carros em movimento. Portanto, o tempo necessário para que o algoritmo processe uma imagem e retorne o resultado deve ser mínimo.

3.3.2 Teste do algoritmo na placa Rock 3A

Para realizar a etapa de testes do algoritmo na placa Rock 3A, é necessário entender as características e limitações desse dispositivo. Além disso, escolhas devem ser feitas na etapa de desenvolvimento do algoritmo considerando as características dessa placa, para que não haja problemas de incompatibilidade.

A placa Rock 3A é um computador de placa única (*Single Board Computer* — SBC) desenvolvido pela Radxa, projetado para aplicações de inteligência artificial e processamento de imagens. Esta placa foi escolhida para validação devido ao seu custo acessível e às suas capacidades de processamento de IA. A figura 10 mostra a vista superior da placa Rock 3A.

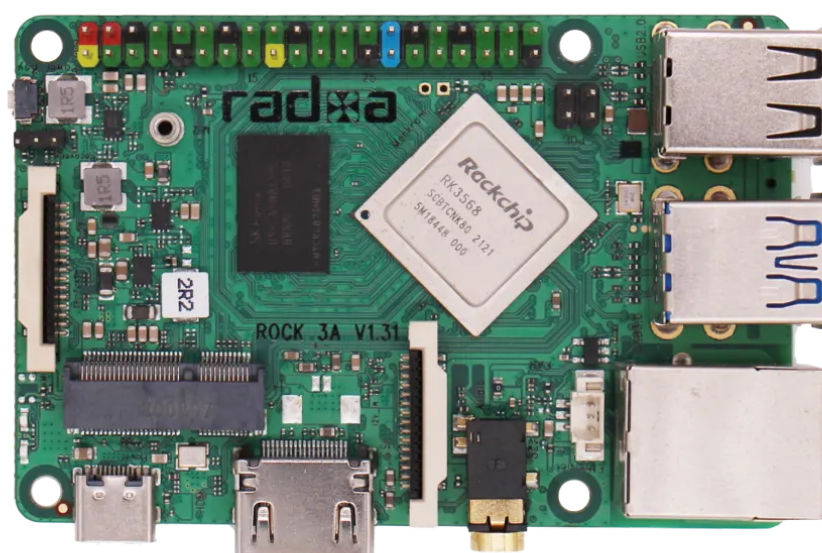


Figura 10 – Placa Radxa ROCK 3A

As principais especificações técnicas da Rock 3A são:

- **Processador:** Rockchip RK3568 quad-core ARM Cortex-A55 de até 2,0 GHz;
- **NPU:** unidade de processamento neural de 0,8 TOPs (*Tera Operations Per Second*);
- **Memória RAM:** disponível em versões de 2 GB, 4 GB e 8 GB LPDDR4;
- **Armazenamento:** slot para cartão microSD e conector eMMC;
- **Conectividade:** Wi-Fi 802.11 b/g/n/ac, Bluetooth 5.0, Ethernet Gigabit;
- **Interfaces:** múltiplas portas USB, HDMI e GPIO de 40 pinos;
- **Suporte a câmeras:** interface MIPI CSI para conexão de câmeras.

A presença da NPU de 0,8 TOPs é particularmente relevante para este projeto, pois permite a aceleração de operações de redes neurais, melhorando significativamente a performance do algoritmo de leitura de placas em comparação com o processamento baseado puramente em CPU.

As limitações da placa que devem ser consideradas no desenvolvimento incluem:

- **Capacidade de processamento limitada:** comparada a soluções mais caras, a Rock 3A possui recursos computacionais restritos;
- **Memória restrita:** dependendo da versão, a quantidade de RAM pode ser um fator limitante para modelos de IA mais complexos;
- **Compatibilidade de software:** nem todos os *frameworks* de IA possuem suporte otimizado para a arquitetura ARM e para a NPU específica desse dispositivo.

A escolha da Rock 3A para este projeto se justifica pela necessidade de validar uma solução de baixo custo que possa ser viável comercialmente, mantendo um equilíbrio entre performance e custo-benefício para aplicações de leitura automática de placas veiculares.

3.3.2.1 NPU da placa Rock 3A

A NPU da Rock 3A é otimizada para operações de inferência, não sendo adequada para treinamento de modelos. Ela oferece melhor performance quando utilizada com modelos quantizados para *uint8*, formato que reduz significativamente o uso de memória e acelera as operações matemáticas, mantendo precisão aceitável para a maioria das aplicações.

Para utilizar a NPU da Rock 3A, é necessário trabalhar com o *framework* RKNN (*Rockchip Neural Network*), que é o SDK (*Software Development Kit*) oficial fornecido pela Rockchip para desenvolvimento de aplicações de inteligência artificial em seus processadores. O RKNN oferece ferramentas para conversão de modelos treinados em *frameworks* populares, como TensorFlow, PyTorch e ONNX, para o formato nativo da NPU.

Os principais requisitos para executar modelos de redes neurais na NPU da Rock 3A são:

- **Conversão de formato:** os modelos devem ser convertidos para o formato RKNN utilizando o *toolkit* oficial;
- **Quantização:** os modelos devem ser quantizados para *uint8* para otimizar a performance na NPU;
- **Compatibilidade de operações:** nem todas as operações de redes neurais são suportadas pela NPU, sendo necessário verificar a compatibilidade das camadas utilizadas;
- **Limitações de memória:** o modelo convertido deve respeitar as limitações de memória da NPU;
- **Formato de entrada:** as imagens de entrada devem estar no formato e na resolução adequados para o modelo convertido.

A utilização da NPU oferece vantagens significativas em termos de eficiência energética e velocidade de inferência quando comparada ao processamento baseado puramente em CPU, tornando-a adequada para aplicações que requerem processamento de imagens em tempo real com restrições de energia e custo.

3.4 REQUISITOS DA SOLUÇÃO

Considerando as características e especificações do projeto discutidas até então, os requisitos funcionais e não funcionais podem ser definidos como se segue.

3.4.1 RF1: software de leitura automática de placas

O *software* deve ser capaz de receber imagens de veículos automotores, como carros, caminhões e ônibus, analisá-las e retornar os caracteres da placa do veículo contido na imagem.

3.4.1.1 Requisitos não funcionais

Código	Nome	Descrição
NF1.1	Acurácia	O percentual de placas inferidas corretamente deve ser suficiente para que a ferramenta seja útil. Preferencialmente acima de 80%.
NF1.2	Velocidade	O tempo necessário para realizar a inferência deve ser apropriado para uma aplicação de tempo real, não superior a 1 segundo por imagem.
NF1.3	Flexibilidade	O sistema deve funcionar tanto para placas do modelo antigo quanto para placas do modelo novo (Mercosul), abrangendo todas as variações possíveis dentro desses modelos.
NF1.4	Especificidade	O algoritmo deve ser capaz de identificar o modelo da placa, entre o padrão antigo e o novo (Mercosul).
NF1.5	Compatibilidade	O <i>software</i> deve ser compatível com a placa Rock 3A.

Tabela 1 – Requisitos não funcionais de RF1

3.4.2 RF2: execução na placa Rock 3A

O algoritmo de leitura automática de placas de veículos deve ser executado na placa Rock 3A da Radxa.

3.4.2.1 Requisitos não funcionais

Código	Nome	Descrição
NF2.1	<i>Neural Processing Unit</i>	A execução dos modelos de redes neurais na placa Rock 3A deve ser realizada na NPU.
NF2.2	Quantização	Os modelos executados na NPU devem ser quantizados para <i>uint8</i> para otimizar o tempo de inferência.

Tabela 2 – Requisitos não funcionais de RF2

4 SOLUÇÃO PROPOSTA

Neste capítulo é apresentada a solução proposta para o problema descrito no capítulo anterior. São discutidas a metodologia de desenvolvimento adotada, a análise dos dados utilizados e a arquitetura geral do *pipeline* de leitura de placas, incluindo as abordagens propostas para cada uma de suas etapas e a estratégia para execução na placa Rock 3A.

4.1 METODOLOGIA DE DESENVOLVIMENTO

A metodologia de desenvolvimento da solução segue o padrão CRISP-DM, discutido no capítulo 2, que trata o projeto como um problema de mineração de dados. O problema de negócio já foi discutido e bem definido no capítulo 3, constituindo a primeira etapa do processo. As demais etapas do padrão aplicadas a este projeto são descritas a seguir.

Entendimento dos dados: antes de desenvolver qualquer modelo, é necessário explorar as características tanto do *dataset* utilizado quanto do objeto de interesse, as placas de veículos. Esse estudo é apresentado na próxima seção deste capítulo.

Preparação dos dados: a preparação dos dados envolve duas etapas distintas, pois este projeto lida com dois conjuntos de dados, sendo o segundo derivado do primeiro. O primeiro conjunto é composto por imagens de carros com anotações das placas, e sua preparação envolve separar o *dataset* em conjuntos de treino, validação e teste, além de organizá-los no formato exigido pelo algoritmo de treinamento do modelo de detecção. A segunda etapa da preparação ocorre posteriormente e consiste na criação de um *dataset* de imagens de caracteres e de um *dataset* de imagens de placas completas, voltados para o treinamento dos modelos de classificação. Para isso, é necessário que a etapa de modelagem já tenha avançado, de modo que preparação dos dados e modelagem se alternam.

Modelagem: a modelagem pode ser dividida em três etapas principais. A primeira é a seleção e o treinamento de um modelo de detecção de objetos para localizar e classificar as placas. A segunda, que depende da primeira, é o desenvolvimento de um algoritmo de extração de caracteres a partir das placas detectadas, utilizando técnicas clássicas de visão computacional. A terceira é o treinamento de modelos de classificação de caracteres, que identificam cada caractere extraído.

Avaliação: a avaliação consiste em executar o *pipeline* completo sobre o conjunto de teste e medir tanto a acurácia das inferências quanto o tempo de processamento. Os testes são realizados no computador de desenvolvimento e na placa Rock 3A, cobrindo execução em CPU, GPU e NPU.

Deployment: neste projeto, o *deployment* corresponde à elaboração de um relatório dos resultados obtidos, que valida o algoritmo e o *hardware* como viáveis ou

não para o desenvolvimento de um produto de leitura automática de placas veiculares. Esse relatório é apresentado no capítulo 5.

4.2 DESCRIÇÃO DOS DADOS

Nesta seção são exploradas as características dos dados utilizados no projeto, que são essenciais para obtenção de bons resultados de modelagem. São exploradas as características do objeto de interesse, as placas veiculares, e também as características específicas do *dataset* utilizado.

4.2.1 Características das placas veiculares

O sistema de placas de carros do Brasil passou por diversas alterações ao longo dos anos, sendo que a mais significativa foi a adoção do padrão Mercosul, em 2018. Desde então, dois padrões de placas veiculares coexistem: o padrão Mercosul e o padrão anterior, de 1990, que segue em circulação embora não sejam mais emitidas novas placas nesse formato.

O padrão antigo, iniciado em 1990, é alfanumérico de 7 caracteres, seguindo o formato ABC-1234 — três letras seguidas de um número de 4 dígitos, com um ponto ou hífen separando os dois blocos.

As placas desse padrão costumam ter fundo cinza metálico com caracteres pretos (figura 11), mas existem exceções importantes. Algumas categorias de veículos utilizam placas de fundo colorido e caracteres brancos, sendo a mais comum a placa vermelha utilizada em veículos de transporte pago (figura 12).



Figura 11 – Modelo de placa veicular do padrão de 1990



Figura 12 – Modelo de placa veicular do padrão de 1990 para veículos de transporte pago

As placas do padrão Mercosul utilizam uma sequência alfanumérica diferente, no formato ABC1D23 — quatro letras intercaladas com três dígitos, sendo a quarta letra posicionada entre dois grupos numéricos. Esse formato permite um número maior de combinações que o anterior.

As placas Mercosul possuem sempre fundo branco, enquanto a cor dos caracteres varia conforme a modalidade do veículo, sendo preto o mais comum (figura 13).



Figura 13 – Modelo de placa veicular do padrão Mercosul

Ambos os padrões incluem informações adicionais na parte superior da placa: o padrão de 1990 exibe a cidade e a UF do veículo, enquanto o Mercosul exibe o país. As placas Mercosul também incluem símbolos e selos de autenticidade. Os dois modelos possuem margens ao redor do texto principal.

4.2.2 Características do *dataset* utilizado

O conjunto de dados utilizado foi o RodoSol-ALPR (Laroca et al., 2022), composto por 10.000 imagens de carros e 10.000 imagens de motocicletas com resolução de 1.280×720 px. As imagens são classificadas por tipo de veículo (carro ou moto) e por padrão de placa (Brasil ou Mercosul), com distribuição igual entre os dois padrões em ambas as categorias. Neste projeto, apenas as imagens de carros foram utilizadas.

As imagens foram capturadas por câmeras estáticas em um pedágio na Rodovia do Sol, no Espírito Santo, e incluem variações de iluminação, presença de chuva e diferentes distâncias do veículo à câmera.

Cada imagem é acompanhada de um arquivo de anotação no seguinte formato:

```
type: car  
plate: ODE2510  
layout: Brazilian  
corners: 558,438 687,439 687,482 558,481
```

A primeira linha especifica o tipo de veículo, a segunda os caracteres da placa, a terceira o padrão da placa (*Brazilian* ou *Mercosur*) e a quarta as coordenadas dos quatro cantos da placa em pixels.

A figura 14 mostra um exemplo de imagem do *dataset* com as marcações dos quatro cantos da placa obtidas do arquivo de anotação.



Figura 14 – Exemplo de imagem do *dataset* com as marcações dos quatro cantos da placa

4.3 PIPELINE DE LEITURA DE PLACAS

O objetivo do *pipeline* é receber uma imagem de um veículo e devolver a sequência de caracteres correspondente à sua placa de identificação. Essa tarefa não é um problema trivial de visão computacional e dificilmente pode ser resolvida alimentando as imagens diretamente a um único modelo de redes neurais. A complexidade se dá principalmente pelos seguintes fatores:

- A placa pode estar localizada em diferentes regiões da imagem e apresentar tamanhos variados;

- O número de placas possíveis é da ordem de $4,6 \times 10^8$ para o padrão Mercosul e $1,8 \times 10^8$ para o padrão antigo, inviabilizando uma abordagem de classificação direta em que cada placa seja uma classe distinta;
- A maior parte do conteúdo de cada imagem é ruído irrelevante para a leitura da placa.

Por conta dessa complexidade, o problema requer uma solução estruturada em etapas. O *pipeline* proposto é dividido em três blocos sequenciais:

1. **Deteção e classificação da placa:** este bloco localiza a placa na imagem, identifica seu padrão (Brasil ou Mercosul) e determina as coordenadas dos seus quatro cantos, permitindo que todo o resto da imagem seja descartado nas etapas seguintes.
2. **Processamento da placa:** com base nas informações obtidas pelo bloco anterior, este bloco aplica transformações na imagem da placa para remover ruído, corrigir distorções e padronizar o formato, facilitando a leitura dos caracteres.
3. **Leitura da placa:** o último bloco recebe a imagem processada e classifica os caracteres da placa, produzindo como saída a sequência de caracteres identificada.

Para o bloco de leitura da placa, duas abordagens distintas são propostas e avaliadas, conforme descrito nas seções a seguir.

4.3.1 Deteção e classificação da placa

Para a tarefa de deteção e classificação da placa, a solução proposta é utilizar um modelo de deteção de objetos e pontos-chave da família YOLO, desenvolvida pela Ultralytics.

Os modelos YOLO são reconhecidos por sua velocidade e precisão na deteção de objetos em uma única etapa, sendo amplamente utilizados em aplicações de tempo real. A variante com deteção de pontos-chave (*pose*) é particularmente adequada para este projeto, pois permite detectar simultaneamente a *bounding box* da placa, sua classe (Brasil ou Mercosul) e as coordenadas dos seus quatro cantos, que são necessárias para a correção de perspectiva na etapa seguinte.

4.3.2 Processamento da placa

Uma vez detectada a placa e conhecidas as coordenadas dos seus quatro cantos, uma sequência de transformações é aplicada à imagem com o objetivo de remover o máximo de ruído e padronizar o formato, entregando para a etapa seguinte uma imagem mais limpa e informativa. As transformações propostas são:

1. Conversão para escala de cinza, eliminando informações de cor desnecessárias para a leitura dos caracteres;
2. Extração da região da placa com base na *bounding box*;
3. Correção de perspectiva, utilizando os quatro cantos detectados para retificar rotações e distorções;
4. Remoção das bordas da placa, descartando regiões que não contêm caracteres;
5. Binarização adaptativa, convertendo a imagem em preto e branco para destacar os caracteres;
6. Separação dos caracteres em imagens individuais;
7. Limpeza das imagens de caracteres, removendo pixels de ruído desconectados do caractere principal.

4.3.3 Leitura da placa: caracteres isolados

Na primeira abordagem proposta para a leitura da placa, os caracteres extraídos individualmente pelo bloco de processamento são classificados por redes neurais convolucionais especializadas. Como a posição de cada caractere na placa determina se ele é uma letra ou um número, e como o padrão da placa é conhecido desde a etapa de detecção, é possível utilizar modelos distintos para cada combinação de tipo de caractere (letra ou número) e padrão de placa (Brasil ou Mercosul), resultando em quatro modelos de classificação. Essa especialização simplifica o problema de cada modelo individualmente e tende a produzir classificadores mais precisos.

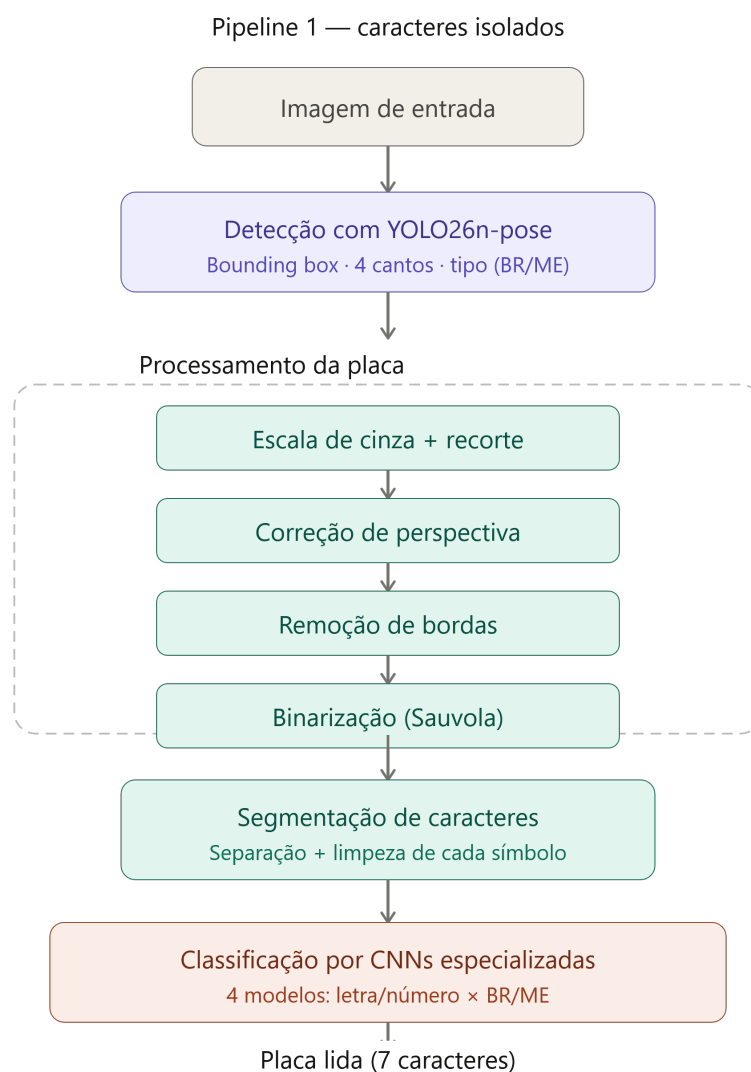


Figura 15 – Diagrama do *pipeline* de classificação de caracteres isolados.

4.3.4 Leitura da placa: extração e classificação simultâneas

Na segunda abordagem, um único modelo mais robusto recebe diretamente a imagem da placa — após a correção de perspectiva — e classifica todos os seus caracteres de uma só vez, sem necessidade das etapas de binarização, separação e limpeza de caracteres. Essa abordagem é mais simples do ponto de vista do *pipeline* e elimina os possíveis erros introduzidos pelo algoritmo de segmentação de caracteres, mas exige um modelo de maior capacidade. São treinados dois modelos dessa modalidade, um para cada padrão de placa.

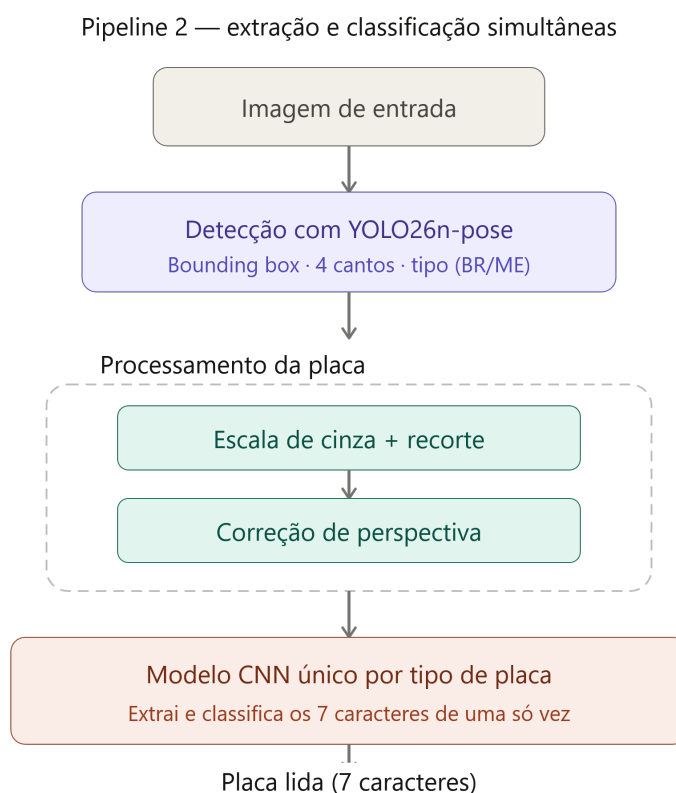


Figura 16 – Diagrama do *pipeline* de extração e classificação simultânea.

4.4 EXECUÇÃO NA PLACA ROCK 3A

Para que os modelos de redes neurais possam ser executados na NPU da placa Rock 3A, é necessário convertê-los do formato de treinamento para o formato RKNN, compatível com o chip Rockchip. Esse processo de conversão também permite a quantização dos modelos para *uint8*, o que reduz o consumo de memória e melhora a performance na NPU.

4.5 METODOLOGIA DE TESTE

Os testes são realizados com base em um conjunto de dados separado no início do projeto, que não é utilizado em nenhuma etapa de desenvolvimento dos modelos, evitando assim viés nos resultados. As métricas avaliadas são:

- Percentual de placas inferidas corretamente sobre o total de imagens do conjunto de teste;
- Tempo de inferência de cada etapa do *pipeline*: detecção, processamento e leitura.

Os testes são conduzidos tanto no computador de desenvolvimento quanto na placa Rock 3A, e contemplam as duas abordagens de *pipeline*, além das diferentes plataformas de execução disponíveis em cada dispositivo (CPU, GPU e NPU).

5 DESENVOLVIMENTO E ANÁLISE DOS RESULTADOS OBTIDOS

Neste capítulo será discutido o que foi efetivamente feito para realização da solução proposta no capítulo 4, assim como a concretização da metodologia utilizada. Também são apresentadas as tecnologias utilizadas e problemas encontrados.

As etapas de desenvolvimento foram:

1. Preparação dos dados
2. Treinamento do modelo de detecção de placas
3. Implementação do algoritmo de extração de caracteres
4. Criação dos *datasets* de caracteres e de placas
5. Treinamento dos modelos de classificação de caracteres
6. Quantização e transformação dos modelos para formato compatível com o *hardware*
7. Preparação do *hardware*
8. Realização dos testes

5.1 PREPARAÇÃO DOS DADOS

O primeiro passo da preparação dos dados foi a separação em conjuntos de treino, validação e teste, cada qual com um propósito diferente:

- **Treino:** O conjunto de treino é utilizado para ajustar os modelos, que irão se adaptar para minimizar o erro de inferência nesses dados.
- **Validação:** Este conjunto é utilizado para, durante o treinamento dos modelos, garantir que o ajuste feito com os dados de treino se generalizem para dados não utilizados no treinamento.
- **Teste:** Estes dados não são utilizados em nenhum momento durante o desenvolvimento e ajustes dos modelos. Sua função é serem utilizados nos testes finais para verificar a eficácia do modelo desenvolvido.

O *dataset* original continha 10 mil imagens de carros, sendo que destas 5 mil eram com placas do tipo Brasil e 5 mil do tipo Mercosul. Os dados foram então divididos da seguinte forma: 70% para o conjunto de treino, 20% para o conjunto de validação e 10% para o conjunto de teste. Os dados colocados em cada conjunto foram selecionados aleatoriamente, mas garantindo que cada conjunto teria 50% de imagens com placas Mercosul e 50% com placas Brasil.

5.1.1 Organização dos dados

O treinamento do modelo de detecção de placas foi realizado com a biblioteca Ultralytics do python. Essa biblioteca, além de oferecer diversos modelos da família YOLO, já inclui o algoritmo de treinamento, necessitando apenas que o usuário deixe os dados organizados em um formato específico. Esse formato inclui deixar os dados de treino e validação em pastas separadas, além de escrever os arquivos de anotações em um formato específico.

Os arquivos de anotações são os arquivos que contêm as informações das *bounding boxes* e pontos-chave das imagens. O Ultralytics exige que eles sejam arquivos do tipo txt com o mesmo nome do arquivo de imagem ao qual se referem. Por exemplo, a imagem *img_000001.jpg* deve ter um arquivo *img_000001.txt* correspondente que contém as anotações daquela imagem. O arquivo de anotações pode ter múltiplas linhas, cada uma correspondendo a um objeto na imagem. Cada linha é composta da seguinte forma:

```
<classe> <x centro> <y centro> <largura> <altura> <x ponto 1> <y ponto 1>  
<visibilidade ponto 1> ... <x ponto n> <y ponto n> <visibilidade ponto n>
```

A classe é o número da classe daquele objeto. No caso do projeto existem duas classes: placa Brasil (0) e placa Mercosul (1). O *x centro* e *y centro* correspondem às coordenadas (*x*, *y*) do centro do objeto. A largura e altura, como o nome sugere, são a largura e altura do objeto. Por fim, cada ponto-chave é composto por três informações: coordenada *x*, coordenada *y* e visibilidade. Como em todas as imagens a placa está com os quatro cantos visíveis, a visibilidade é sempre 2, que é o valor que representa pontos visíveis. Todas as coordenadas, assim como altura e largura são normalizadas para que fiquem entre 0 e 1.

5.2 MODELO DE DETECÇÃO DE PLACAS

Com os dados preparados nas pastas, basta criar um arquivo de configuração no formato YAML e escrever um simples script em python para iniciar o treinamento do modelo desejado. O arquivo de configuração deve conter as seguintes especificações:

- O caminho para a pasta que contém os dados
- O sub caminho para as pastas que contêm os dados de treino e validação
- O número de classes
- Os nomes das classes
- O formato dos pontos-chave

- Índice espelhado dos pontos-chave

No caso deste projeto o número de classes é 2, cujos nomes foram definidos como *placa_br* e *placa_me*. O formato dos pontos-chave deve ser uma tupla contendo dois valores: o número de pontos-chave por objeto e a dimensão dos pontos-chave, que é 3 (largura, altura, visibilidade). O índice espelhado dos pontos-chave é uma lista de índices de 0 a n , sendo que n é a quantidade de pontos-chave por objeto. A ordenação dos índices deve indicar como os índices devem ser tratados caso a imagem seja espelhada horizontalmente. Isso é necessário pois o Ultralytics faz isso com algumas imagens durante o processo de treinamento.

Para a tarefa de detecção e classificação da placa, a solução proposta foi utilizar um modelo de detecção de objetos da família YOLO, mais especificamente o YOLO26-pose, desenvolvido pela Ultralytics. Os modelos da família YOLO são notórios por serem precisos e rápidos na detecção de objetos. Nessa linha, os modelos YOLO26 foram desenvolvidos pensando especificamente em IA de borda, sendo portanto otimizados para entregar excelente performance com baixa latência e utilizando o mínimo de recursos. Uma das variantes disponíveis do YOLO26 é o YOLO26-pose, que além de realizar detecção de objetos também realiza detecção de pontos-chave na imagem, sendo portanto ideal para o caso de uso desse projeto. O YOLO26-pose ainda possui 5 tamanhos, apresentados na Tabela 3.

Tamanho	Parâmetros (milhões)	Velocidade CPU (ms)	mAPpose 50-95(e2e)
nano	2.9	40.3	57.2
small	10.4	85.3	63.0
medium	21.5	218.0	68.8
large	25.9	275.4	70.4
extra large	57.6	565.4	71.6

Tabela 3 – Tamanhos do modelo de detecção de objetos YOLO26-pose

A métrica mAPpose 50-95(e2e) representa a precisão média para estimativa de pose (pontos chave). Nota-se que quanto maior o modelo mais alta é a precisão, porém mais alta também é a latência. Para este projeto, como estamos lidando com um objeto relativamente simples (placas de carro) e o *hardware* é bastante limitado, foi escolhida a versão nano do modelo, que prioriza velocidade de inferência e economia de recursos.

Com o arquivo de configuração preparado é possível executar o script de treinamento, que é bastante simples. Precisa primeiro importar a biblioteca Ultralytics e criar um objeto da classe YOLO, que recebe como argumento o nome do modelo YOLO desejado, que no caso é *yolo26n-pose*. Em seguida basta chamar o método *train*, indicando o caminho para o arquivo de configuração que foi criado, o tamanho do *batch* desejado, o número de épocas e o dispositivo no qual executar o modelo.

Foram treinadas 100 épocas no dispositivo *cuda* (GPU do computador). O tamanho do *batch* foi deixado como -1, que faz com que o próprio Ultralytics defina o melhor tamanho de *batch* com base na capacidade da GPU. As 100 épocas se mostraram suficientes para ajustar o modelo, que ao final do treinamento estava com um erro bem pequeno e estável, tanto na detecção dos objetos como na classificação e detecção dos pontos-chave. A figura 17 mostra três curvas do erro do modelo ao longo do treinamento. A primeira é o erro do ajuste da *bounding box*, a segunda é o erro dos pontos-chave e a terceira é o erro de classificação.

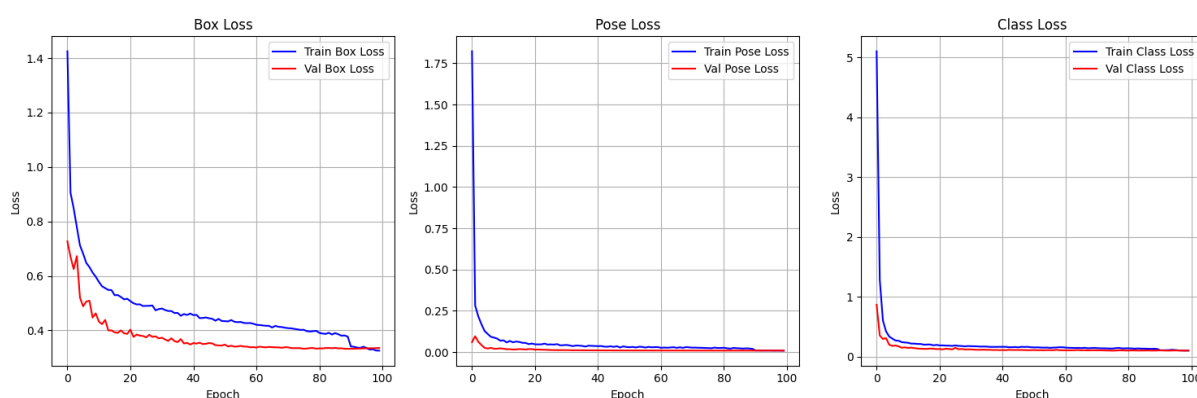


Figura 17 – Curvas de treinamento do modelo de detecção de placas

5.2.1 Exportação do YOLO

Findado o treinamento o Ultralytics automaticamente exporta um arquivo com os pesos do modelo ajustados. Porém esse formato é compatível apenas para realizar inferência com a própria biblioteca Ultralytics. Para obter o modelo em um formato com mais portabilidade, basta criar um novo script, carregar o modelo treinado e utilizar o método *export* para obter o modelo em um arquivo do tipo ONNX. Esse formato permite inferência em diversas plataformas através da biblioteca *onnx runtime*, mas também permite que o modelo seja carregado pelo software que realiza a conversão para o formato RKNN, compatível com a NPU da placa Rock 3A.

5.3 ALGORITMO DE EXTRAÇÃO DE CARACTERES

Após o ajuste do modelo de detecção de placas, foi criado o script de processamento das imagens e extração dos caracteres. Esse script foi feito em python, e utilizou a biblioteca *onnx runtime* para realizar inferência com o modelo de detecção de placas exportado para formato ONNX. As etapas do script são as seguintes:

1. Inferência com o modelo de detecção de placas
2. Criação de uma nova imagem contendo apenas a placa, cortada da imagem original com base na *bounding box* gerada pelo modelo de detecção

3. Correção de perspectiva
4. Remoção das bordas da placa com base no tipo de placa verificado pelo modelo de detecção
5. Binarização da imagem
6. Separação dos caracteres
7. Limpeza das imagens dos caracteres

A seguir será discutida em mais detalhes a implementação de cada etapa do script.

5.3.1 Inferência com o YOLO

O modelo YOLO foi treinado com a biblioteca Ultralytics e exportado para formato ONNX, podendo agora ser utilizado para inferência com a biblioteca do python *onnx runtime*. A implementação da inferência, então, incluiu carregar o arquivo do modelo e criar uma função que recebe a imagem e devolve a inferência. A função não se limita simplesmente a alimentar a imagem ao modelo, pois é necessário algum pré-processamento da imagem antes disso. Primeiramente a imagem deve ser redimensionada para o formato 640×640 px, que é a dimensão utilizada pelo YOLO. Em seguida os valores dos pixels devem ser normalizados para que fiquem entre 0 e 1.

O redimensionamento foi feito com a biblioteca de manipulação de imagens OpenCV, assim como o próprio carregamento da imagem. O OpenCV carrega a imagem como um tensor de dimensão (H,W,C), onde H é a altura da imagem, W é a largura e C é o número de canais, que é 3 para imagens coloridas e 1 para imagens em escala de cinza. Cada valor representa a intensidade de um pixel em um dado canal (para imagens coloridas os canais são vermelho, verde, azul), que é representada por um número inteiro de 0 a 255 (8 *bits*).

O OpenCV oferece uma função para redimensionamento de imagem, já a normalização pode ser feito simplesmente dividindo o tensor por 255. Com isso, a imagem fica pronta para ser alimentada ao modelo.

A última etapa dessa implementação é a interpretação da saída do modelo. Ao alimentar o modelo com a imagem, ele devolve uma saída no formato de uma matriz 300×18 . Cada linha dessa matriz representa uma detecção representada por 18 valores. A única detecção que nos interessa é a primeira, pois as detecções são ordenadas em ordem de confiança. O modelo é obrigado a gerar 300 detecções pela forma como foi construído, mas a maioria delas não representa nenhum objeto. Como nosso objetivo é analisar apenas uma placa por imagem, pegamos apenas a primeira detecção e descartamos as demais, de forma que se houver mais de uma placa, apenas a mais visível será analisada.

Dos 18 valores que representam a detecção a interpretação é a seguinte:

- Os 4 primeiros valores representam a *bounding box* do objeto na seguinte ordem: x_1, y_1, x_2, y_2
- O quinto valor representa a confiança da detecção, que vai de 0 a 1
- A classe do objeto (*placa_br* ou *placa_me*) é representada pelo sexto valor
- As coordenadas dos pontos-chave são os valores seguintes

Como a imagem foi redimensionada para 640×640 é necessário converter as coordenadas dos pontos-chave e da *bounding box* para a dimensão da imagem original. Para isso basta multiplicar as coordenadas x pela largura da imagem original e as coordenadas y pela altura e dividir todas por 640. Assim temos as coordenadas nas dimensões da imagem original.

5.3.2 Criação de imagem da placa

Com base na *bounding box* gerada pelo modelo, é possível cortar a imagem original e gerar uma imagem que contenha apenas a placa detectada. Antes disso, porém, convertamos a imagem para escala de cinza, de forma que seja representada por uma única matriz ($H \times W$). A conversão para escala de cinza é feita pela biblioteca OpenCV. Enquanto as cores poderiam ser úteis para detectar a placa na imagem original, elas já não têm muito propósito a partir desse ponto, pois as placas praticamente não têm cores, e quando têm elas não são relevantes. Por conta disso, as cores são informação desnecessária que só complicaria as etapas posteriores, de forma que é melhor eliminá-las. Ao converter a imagem para escala de cinza (figura 18) ela passa a ser representada por uma matriz $h \times w$ (sendo h a altura da imagem e w a largura), ao invés de três matrizes, como era quando tinha cores.



Figura 18 – Imagem em escala de cinza

Depois disso, basta criar uma nova matriz apenas com os pixels cujas coordenadas (x, y) obedecem às limitações: $x_1 \leq x \leq x_2$ e $y_1 \leq y \leq y_2$, sendo que x_1, x_2, y_1, y_2 são os limites da *bounding box* na imagem original. A *bounding box* de um objeto em uma imagem nada mais é do que os limites superior, inferior, direito e esquerdo do objeto. Com essas quatro informações é possível formar um retângulo que, idealmente, contém o objeto inteiro sem ser maior do que o necessário. Uma das saídas do modelo de detecção de objetos é a *bounding box* de cada objeto detectado na imagem. É importante notar que é possível que mais de um objeto seja detectado em uma dada imagem, caso no qual apenas a melhor detecção será utilizada, ou seja, a que tiver a confiança mais alta. A figura 19 exemplifica o processo de extração da placa com base na *bounding box* inferida pelo modelo de detecção de objetos.

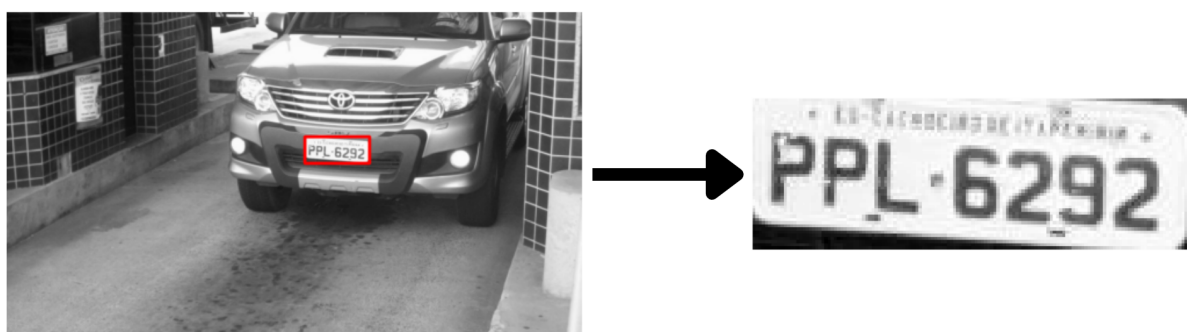


Figura 19 – Extração da placa com base na bounding box

5.3.3 Correção de perspectiva

A correção de perspectiva é realizada pela biblioteca OpenCV em uma sequência de etapas:

1. Definição das coordenadas dos pontos antes da transformação
2. Definição das coordenadas dos pontos depois da transformação
3. Geração da matriz de transformação
4. Transformação com base na imagem e na matriz

Os pontos antes da transformação nada mais são do que as coordenadas dos pontos-chave encontrados pelo YOLO. A única questão é que eles precisam ser transformados para que a referência das coordenadas seja a imagem cortada, não a imagem original. A operação para essa transformação foi discutida no capítulo 4. Para realizar isso é necessário primeiro realizar uma transformação nas coordenadas dos cantos para que o ponto de referência deixe de ser a origem da imagem inicial (do

carro inteiro) e passe a ser a origem da imagem da placa. Para isso, basta aplicar as equações:

$$x_2 = x_1 - x_{min} \quad (19)$$

$$y_2 = y_1 - y_{min} \quad (20)$$

Onde (x_1, y_1) são as coordenadas de um ponto referente à imagem do carro e (x_2, y_2) são as coordenadas referentes à imagem recortada da placa. (x_{min}, y_{min}) são os limites inferiores de x e y na *bounding box*.

Os pontos pós transformação nada mais são que os quatro cantos de uma imagem hipotética de dimensões $W_2 \times H_2$. Essa dimensão será a dimensão da imagem após a transformação, que foi definida como 500×200 . Portanto os 4 pontos pós transformação são: $(0, 0)$, $(499, 0)$, $(499, 199)$, $(0, 199)$.

Definidos os pontos pode ser gerada a matriz de transformação com base neles, o que é feito através de uma função do OpenCV. Em seguida, basta utilizar uma outra função do OpeCV para obter a imagem transformada com base na matriz de transformação, na imagem inicial (imagem cortada da placa) e nas dimensões desejadas da imagem final. A figura 20 exemplifica o que acontece com a imagem da placa após a transformação. Além de a placa se ajustar muito melhor na imagem, suas dimensões ficam padronizadas em 500x200 px.

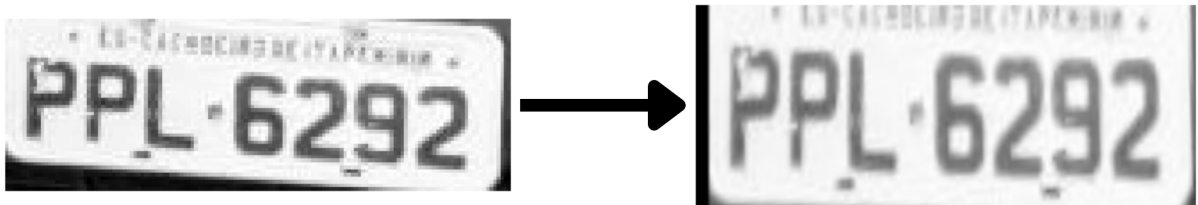


Figura 20 – Transformação de perspectiva com base nos quatro cantos da placa

Com isso, temos uma função que recebe uma imagem com dimensões $(w \times h)$ de uma placa e o *output* do modelo YOLO e devolve uma imagem de dimensões (500×200) com a perspectiva da placa corrigida.

5.3.4 Remoção das bordas e binarização

A remoção das bordas das placas implica basicamente em cortar as imagens de acordo com as dimensões do tipo de placa. Para fazer isso basta limitar os pixels que devem fazer parte da imagem cortada. Considerando as coordenadas (x, y) dos pixels basta seguir o padrão:

$$c_l \leq x \leq 500 - c_r \quad (21)$$

$$c_t \leq y \leq 200 - c_b \quad (22)$$

Sendo que c_l , c_r , c_t e c_b são a quantidade de pixels a cortar na esquerda, direita, topo e parte de baixo da imagem, respectivamente. Esses valores são diferentes para placas do tipo Brasil e Mercosul, como discutido no capítulo 4. Nas placas do modelo Brasil são removidos 40 pixels do topo e 10 pixels da parte de baixo, além de 10 pixels na esquerda e na direita. Já nas placas do modelo Mercosul são removidos 40 pixels do topo, da esquerda e da direita, além de 10 pixels da parte de baixo. As figuras 21 e 22 mostram as medidas removidas das placas.

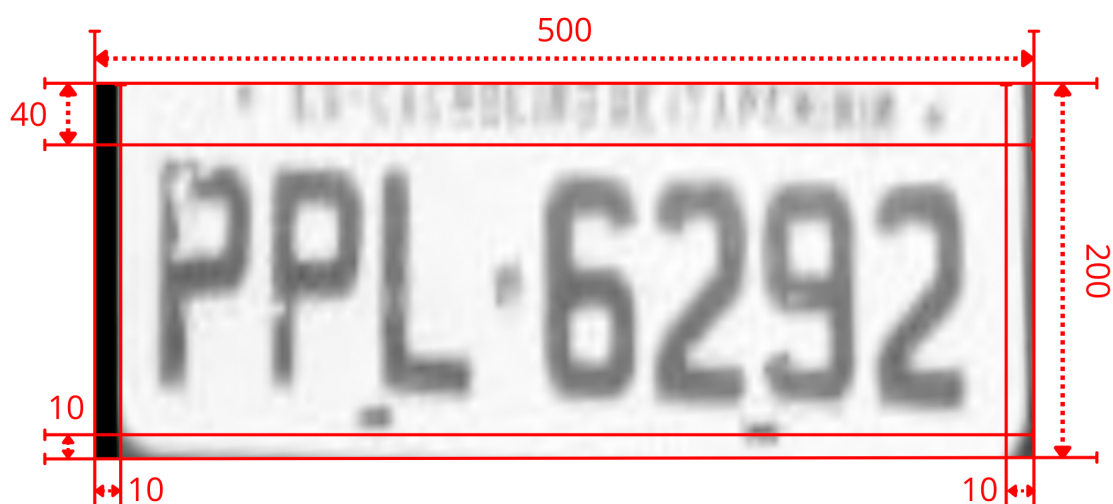


Figura 21 – Medidas para remoção de bordas das placas do padrão antigo

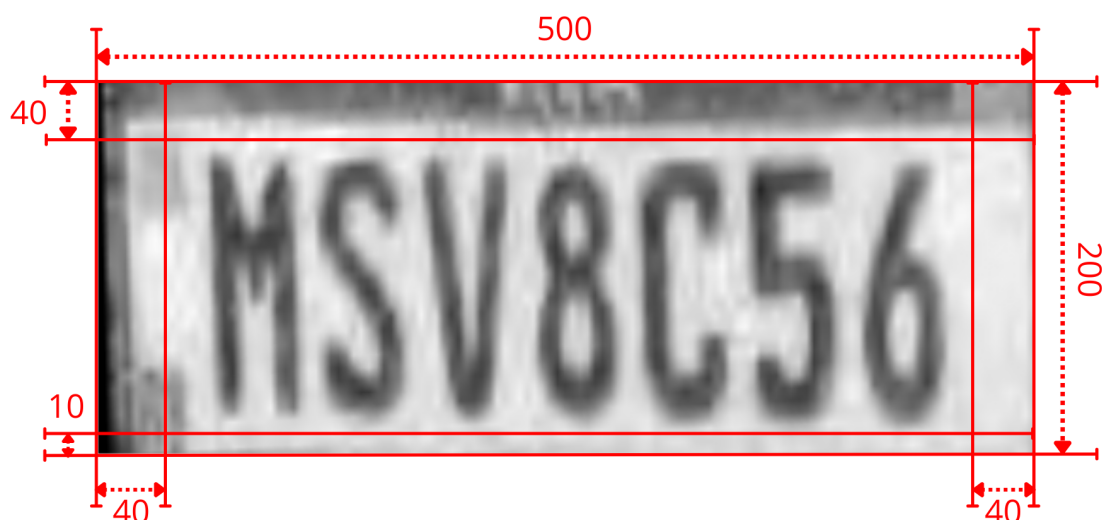


Figura 22 – Medidas para remoção de bordas das placas do padrão Mercosul

A figura 23 demonstra a transformação de remoção de bordas nas duas modalidades de placas. Essa operação ajuda a destacar a parte da imagem que realmente interessa à análise e remover o restante.



Figura 23 – Remoção de bordas das placas

A binarização, que utilizou o método de Sauvola, foi feita utilizando algumas funções diferentes do OpenCV. O primeiro passo foi gerar uma imagem na qual cada pixel era a média dos pixels vizinhos na imagem da placa. Para isso, foi utilizada

uma função do OpenCV que faz exatamente essa tarefa a partir de uma imagem e um tamanho de janela, que para o nosso caso foi escolhido 201 pixels. Esse é um tamanho relativamente grande considerando que a imagem possui 500 pixels de largura, mas o propósito é justamente não permitir flutuação demais para que o resultado seja menos afetado por ruído e capture apenas a variação de luminosidade ao longo da imagem.

Em seguida foi calculado a mesma coisa para o quadrado da imagem (a intensidade de cada pixel foi elevada ao quadrado). Por fim, utilizando simples operações matemáticas, foi calculado o limiar para cada pixel da imagem, e foi gerada uma imagem binarizada na qual cada pixel era 0 ou 1, a depender se o pixel da imagem original naquela posição era menor ou maior que o pixel na matriz de limiar para aquela posição. As equações para cálculo da matriz de limiar foram discutidas mais a fundo no capítulo 2. A figura 24 apresenta um exemplo de imagem resultante deste processo de binarização.



Figura 24 – Imagem de placa binarizada com o método de Sauvola

5.3.5 Separação dos caracteres

A implementação da separação de caracteres da imagem binarizada se baseou na seguinte sequência de eventos:

1. É gerada uma estimativa dos pontos de separação, cada uma representada como uma coordenada x da imagem da placa
2. A função itera por cada um desses pontos, tentando encontrar um ponto de divisão melhor nos seus arredores
3. Os pontos de divisão refinados são utilizados para dividir a imagem em n pedaços, cada um contendo apenas um caractere
4. Cada imagem de caractere é "estampada" em uma imagem preta quadrada

Para funcionar a função precisa dos seguintes parâmetros:

- A imagem a ser separada
- A quantidade de caracteres presentes naquela imagem
- O tamanho da janela na qual um ponto de corte melhor será procurado (padronizado em 40)

A quantidade de caracteres presentes na imagem define o número de separações a ser feito. Isso é importante pois enquanto as placas do tipo Mercosul são separadas de uma só vez, as placas do tipo brasil são primeiro divididas em duas para que depois cada parte seja separada, como já discutido no capítulo 4.

O tamanho da janela define a até quantos pixels de distância do ponto de separação inicialmente estimado será buscado um ponto de separação melhor. A avaliação dos candidatos a pontos de separação é simples: Cada ponto é uma coluna da imagem, sendo então somadas a intensidade de todos os pixels naquela coluna. Como a binarização já foi feita, pixels que não pertencem a nenhum caractere possuem valor 0, enquanto os que pertencem possuem valor 1. Portanto, a coluna cuja soma das intensidades dos pixels for menor (idealmente 0) é melhor para a divisão, pois provavelmente é um espaço vazio entre dois caracteres. A figura 25 mostra as etapas e o resultado da separação de caracteres para os dois modelos de placa.

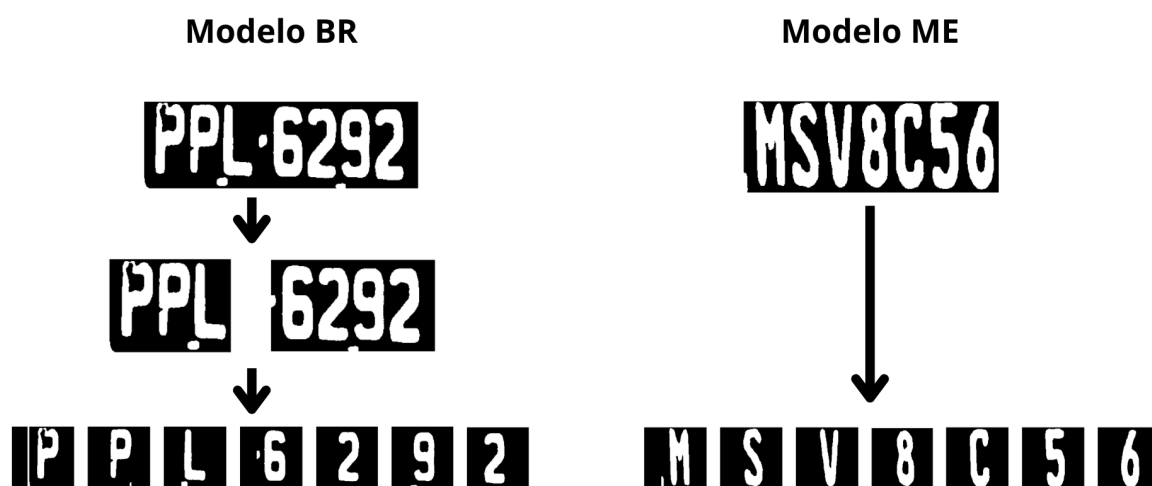


Figura 25 – Separação de caracteres para os dois modelos de placa

Depois de dividir a imagem, porém, cada uma terá uma largura diferente. Isso é então padronizado ao gerar uma imagem quadrada preta $H \times H$ (onde H é a altura da imagem da placa) e "estampando" a imagem com o caractere no meio dessa imagem. Isso garante que todas as imagens de caracteres terão as mesmas dimensões.

5.3.6 Limpeza dos caracteres

Para a limpeza das imagens de caracteres foi utilizada a biblioteca *scikit-image* do Python. Ela oferece uma função que, ao receber uma imagem binarizada, como são as imagens de caracteres, mapeia cada pixel branco (valor 1) como parte de um objeto, com base na continuidade. Todo pixel branco que está adjacente a um outro pixel branco faz parte do mesmo objeto que este. Dessa forma a imagem é dividida em vários objetos, e representada por uma matriz na qual cada pixel é um "rótulo", um valor inteiro que indica a qual objeto pertence. Pixels com valor zero são pixels pretos, portanto não pertencem a objeto nenhum.

O *scikit-image* então oferece uma outra função na qual recebe essa matriz de rótulos e calcula características dos objetos, como altura, largura, área, etc. A única que nos interessa é a área, pois ela é o melhor indicativo de qual é o maior objeto da imagem, que seria o caractere. Com base nisso, então, pixels que não pertencem ao objeto de maior área são eliminados da imagem do caractere, passando a ter valor zero. Isso elimina ruído e deixa a imagem do caractere mais limpa, minimizando a propensão à erros de classificação daquele caractere. Após essa eliminação as imagens passam a conter consideravelmente menos ruído, como pode ser observado na figura 26.

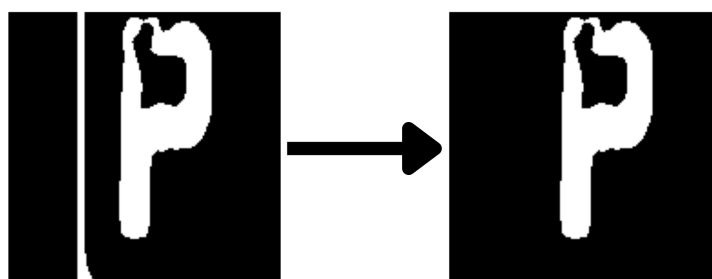


Figura 26 – Exemplo de limpeza de imagem de caractere com base em continuidade de objetos

5.4 CRIAÇÃO DOS DATASETS DE CARACTERES E DE PLACAS

Finalizados todos os algoritmos de processamento das imagens passa a ser possível criar os *datasets* para o treinamento dos modelos de classificação. Como temos duas abordagens distintas de modelos, foram criados dois *datasets*: Um com imagens de caracteres binarizadas e limpas e uma com imagens em escala de cinza de placas com perspectiva corrigida.

Para criar o *dataset* de caracteres todas as imagens do conjunto de treino e validação inicial fora submetidas ao *pipeline* completo desenvolvido até aqui. Passaram pelo modelo YOLO, onde a placa foi detectada, foram cortadas para manter só a

imagem da placa, convertidas para escala de cinza, tiveram sua perspectiva corrigida, passaram pela remoção de bordas, binarização, separação de caracteres e limpeza. Ao final de tudo, cada imagem de caractere foi rotulada com o caractere contido naquela imagem, que era sabido pelo fato de termos nos dados originais as informações das placas dos carros contidos neles. Todas as imagens foram então salvas em pastas de acordo com:

- O tipo de caractere (número ou letra)
- O tipo de placa (Brasil ou Mercosul)

Os nomes das imagens salvas foram compostos pelo caractere representado na imagem, o nome da imagem original da qual o caractere foi extraído e a posição do caractere na placa (0 - 6). O *dataset* de placas teve um processo de criação mais simples. As imagens originais passaram pela detecção, corte, transformação para escala de cinza e correção de perspectiva, apenas. Com isso já se tinham imagens de placas com dimensões padronizadas, que foram então salvas em pastas separadas apenas pelo tipo de placa (Brasil, Mercosul). Os nomes das imagens continham os caracteres da placa e mais um código identificador.

5.5 MODELOS DE CLASSIFICAÇÃO DE CARACTERES

Foram feitos dois treinamentos de modelos de classificação de caracteres. O primeiro de modelos de classificação de caracteres isolados, como os gerados pelo *pipeline* completo de extração de caracteres, e o outro de modelos de extração e classificação, com intenção de classificar os caracteres direto das imagens de placas extraídas e com perspectiva corrigida. Ambos foram realizados utilizando a biblioteca PyTorch e ao final do treinamento exportados para o formato ONNX.

Os treinamentos foram realizados utilizando o seguinte ciclo:

1. *Batch* de imagens é alimentado ao modelo
2. É calculado o *Loss* do *Batch* (*Cross Entropy*)
3. É realizado *backpropagation* para cálculo do gradiente
4. Os pesos do modelo são atualizados

Esses passos são repetidos até que o modelo tenha sido alimentado com todos os dados do *dataset*. É então finalizada a época, o modelo é avaliado e tudo é feito novamente até que o número de épocas determinado inicialmente seja atingido.

5.5.1 Classificação de caracteres isolados

As características específicas do modelo de classificação de caracteres isolados são os seguintes parâmetros de treinamento:

- *Optimizer*: Adam com *learning rate* de 0.001
- Tamanho do *batch*: 32
- Número de épocas: 10

Foram treinados um total de quatro modelos distintos, cada um especializado em um tipo de caractere e um tipo de placa, seguindo a divisão feita na criação do *dataset* de caracteres. Os modelos de classificação de caracteres têm a atribuição de receber uma imagem contendo um único caractere e classificá-lo entre as classes possíveis para aquele modelo. Como já é previamente conhecido, com base na posição dos caracteres, quando um caractere é uma letra ou um número, é possível criar modelos separados para classificação de letras e de números. Além disso, como também é sabido qual o modelo da placa, já que isso é inferido pelo modelo de detecção de objetos, é possível também que cada modelo seja especializado em caracteres do modelo Brasil ou do modelo Mercosul.

A figura 27 mostra o diagrama da rede neural utilizadas na classificação de letras e na classificação de números. A única diferença entre as duas é que a saída da rede de números é um vetor de 10 elementos, representando os 10 algarismos existentes, enquanto a rede de letras é um vetor de 26 valores, representando cada letra do alfabeto.

Os tipos de camadas utilizadas na rede neural seguindo a notação das figuras foram:

- **Conv2d**: Camada convolucional, definida pelo tamanho da saída, dimensões do *kernel* (k) e *padding* (p);
- **Linear**: Camada linear *fully connected*, definida pela dimensão da saída.

Além disso foram utilizadas as operações:

- **Relu**: Rectified Linear Unit;
- **MaxPool2d**: Max pooling, definido pelas dimensões do *kernel* (k) e *stride*;
- **Reshape**: Modificação da forma do tensor, definido pela forma de saída.

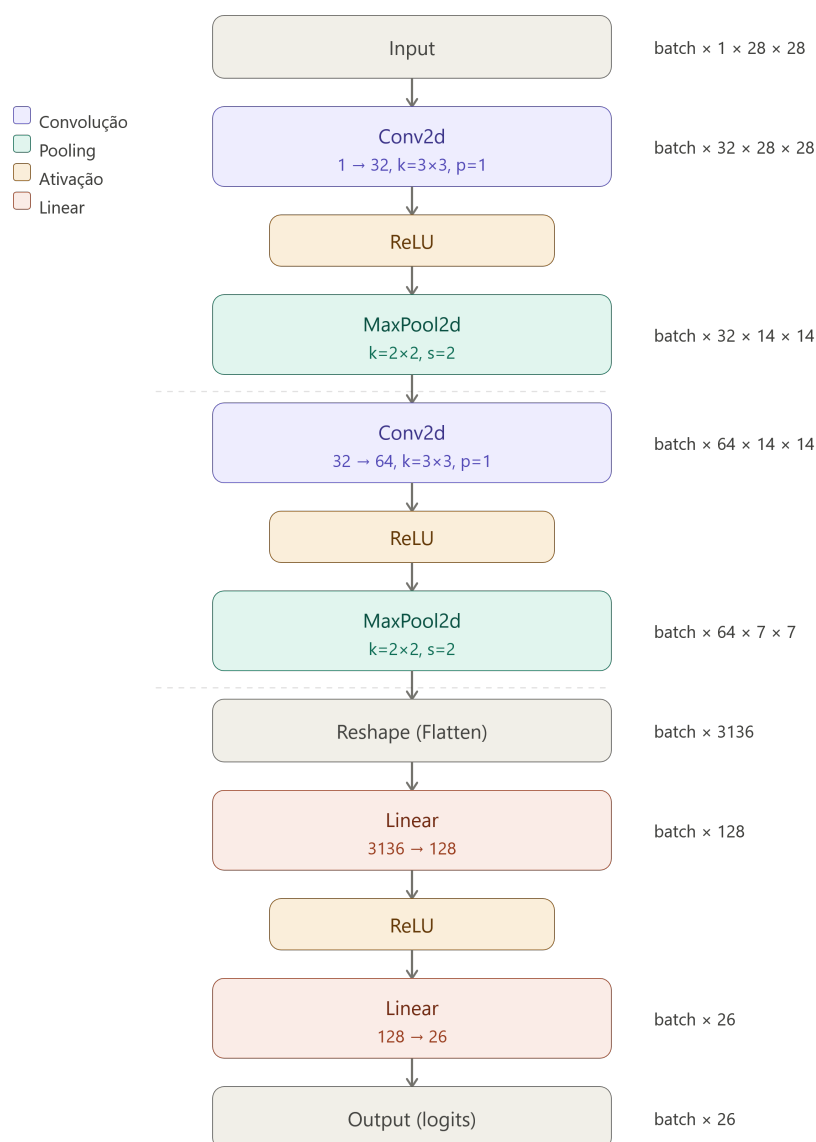


Figura 27 – Diagrama da rede neural de classificação de caracteres

As figuras 28, 29, 30 e 31 mostram as curvas do *Loss* de treinamento e validação, além da acurácia de validação de cada modelo ao longo das épocas de treinamento. Esses dados foram coletados ao fim de cada época com intenção de registrar a evolução do modelo ao longo do treinamento.

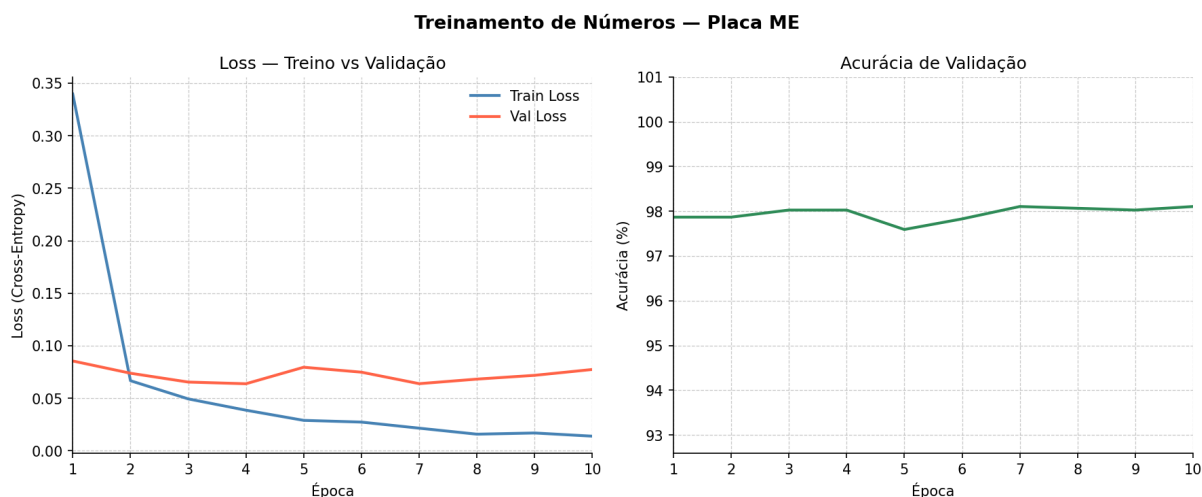


Figura 28 – Curvas de treinamento do modelo de classificação de números em placas do tipo Mercosul

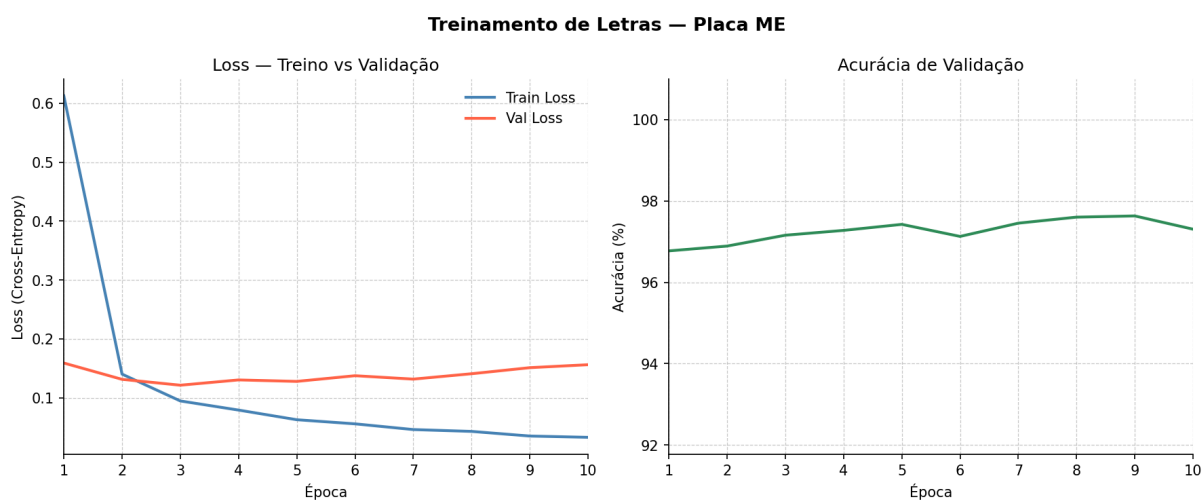


Figura 29 – Curvas de treinamento do modelo de classificação de letras em placas do tipo Mercosul

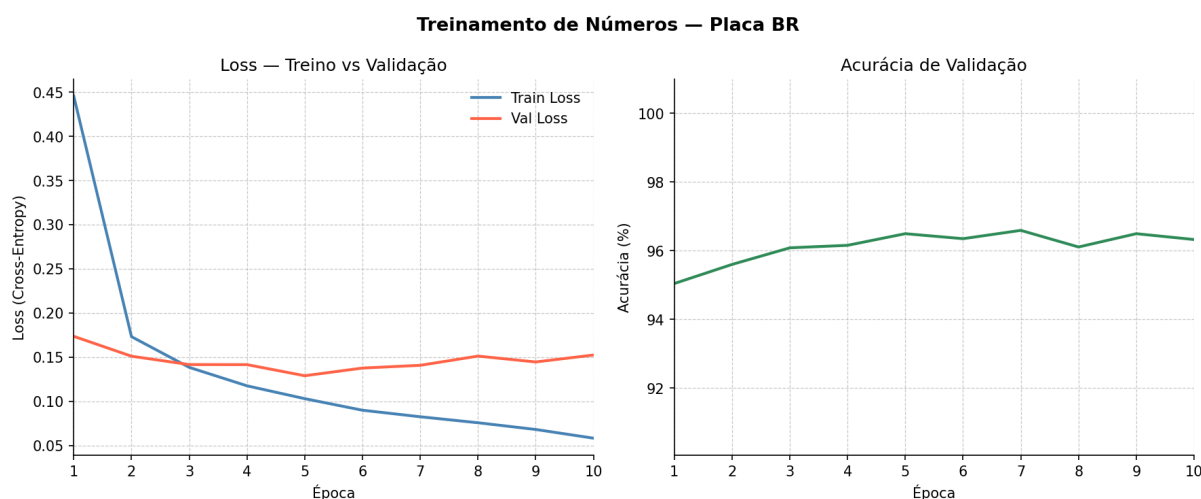


Figura 30 – Curvas de treinamento do modelo de classificação de números em placas do tipo Brasil

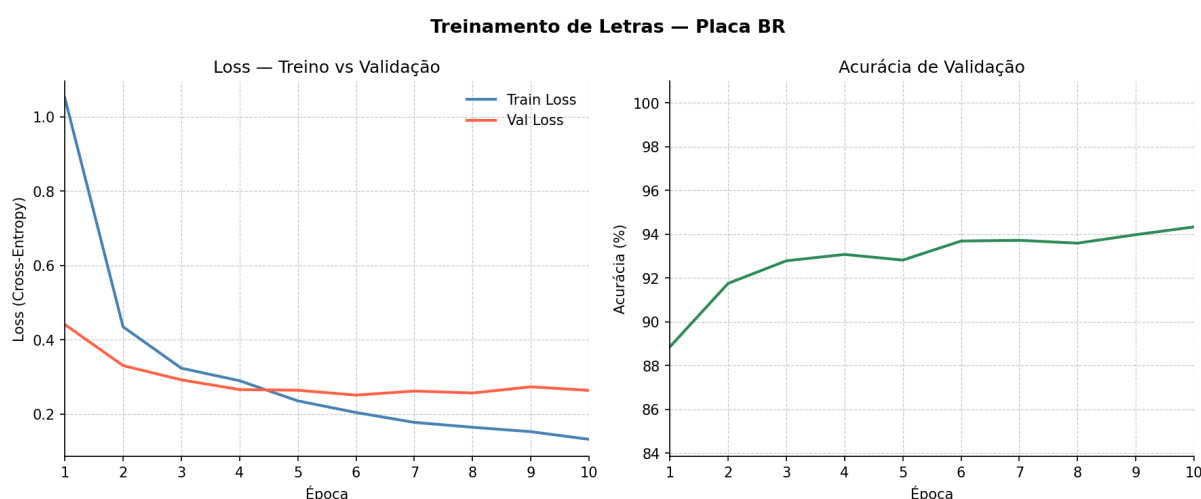


Figura 31 – Curvas de treinamento do modelo de classificação de letras em placas do tipo Brasil

5.5.2 Extração e classificação simultânea

O treinamento dos modelos de extração e classificação seguiu um processo parecido. Esse modelo, diferente dos modelos de classificação de caracteres isolados, recebe uma imagem de placa e classifica todos os seus caracteres de uma só vez. O objetivo é que o modelo aprenda a identificar a posição dos caracteres e ao mesmo tempo classificá-los.

Esses modelos devem extrair e classificar os caracteres diretamente da placa cortada da imagem original após a detecção pelo YOLO. A única transformação a ser feita na placa antes de alimentar para esse modelo é a transformação de perspectiva, pois esta garante uma padronização muito maior das placas enviadas ao modelo. A saída desses modelos é um conjunto de 7 vetores, cada um representando um dos

caracteres da placa. Cada vetor é composto por 10 ou 26 valores, a depender se aquele vetor representa um número ou uma letra, o que depende da posição do caractere que aquele vetor representa.

Como as placas dos modelos Brasil e Mercosul possuem configurações diferentes, devem ser feitos dois modelos, um para cada tipo de placa. Esses modelos possuem uma arquitetura consideravelmente mais complexa que os modelos de classificação de caracteres isolados, podendo ser dividida em três blocos principais:

- **Backbone:**

O *backbone* é responsável pela extração de características da imagem e recebe como entrada um tensor de dimensão $(B, 1, 32, 100)$, onde B é o tamanho do *batch*. Ele é composto por uma sequência de blocos convolucionais e operações de *max pooling* que progressivamente reduzem a resolução espacial e aumentam o número de canais. O bloco convolucional base utilizado, denominado *ConvBN-ReLU*, é formado por uma camada *Conv2d* seguida de *Batch Normalization* e ativação ReLU.

Além disso, o *backbone* faz uso de blocos residuais (figura 32). Cada bloco residual é composto por um bloco *ConvBNReLU* seguido de uma convolução e uma normalização de *batch*, cuja saída é somada à entrada do bloco antes da ativação ReLU final. Essa conexão residual facilita o fluxo do gradiente durante o treinamento e permite que a rede aprenda transformações incrementais em vez de reconstruir a representação do zero em cada camada.

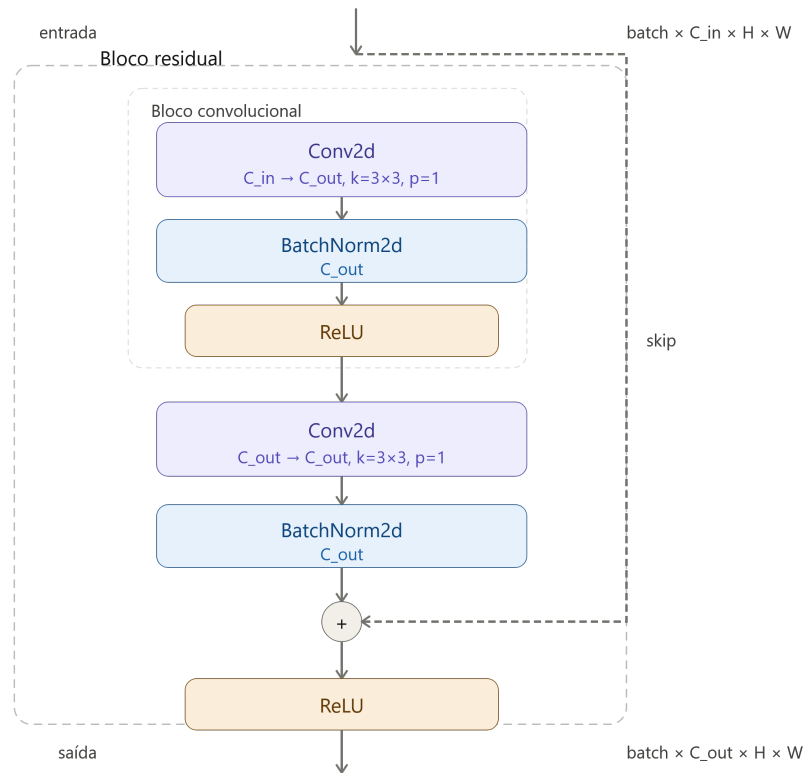


Figura 32 – Diagrama do bloco residual

A sequência completa do *backbone* é apresentada na tabela 4, que mostra cada operação e as dimensões do tensor após cada uma delas.

Operação	Dimensão de saída
Entrada	$(B, 1, 32, 100)$
<i>ConvBNReLU</i> (1 → 32)	$(B, 32, 32, 100)$
<i>ConvBNReLU</i> (32 → 32)	$(B, 32, 32, 100)$
<i>MaxPool2d</i> (2×2)	$(B, 32, 16, 50)$
<i>ConvBNReLU</i> (32 → 64)	$(B, 64, 16, 50)$
<i>ResidualBlock</i> (64)	$(B, 64, 16, 50)$
<i>MaxPool2d</i> (2×2)	$(B, 64, 8, 25)$
<i>ConvBNReLU</i> (64 → 128)	$(B, 128, 8, 25)$
<i>ResidualBlock</i> (128)	$(B, 128, 8, 25)$
<i>MaxPool2d</i> (2×2, pad = 1)	$(B, 128, 4, 13)$
<i>ConvBNReLU</i> (128 → 256)	$(B, 256, 4, 13)$
<i>ResidualBlock</i> (256)	$(B, 256, 4, 13)$
<i>MaxPool2d</i> (2×2)	$(B, 256, 2, 6)$
<i>ConvBNReLU</i> (256 → 256)	$(B, 256, 2, 6)$
<i>AdaptiveAvgPool2d</i> (1×7)	$(B, 256, 1, 7)$

Tabela 4 – Sequência de operações do *backbone* e dimensões dos tensores

- **Average Pooling adaptativo:**

A última operação do *backbone* é um *AdaptiveAvgPool2d* com saída de dimensão (1×7) , que reduz a dimensão espacial vertical para 1 e a horizontal para 7 — uma coluna por caractere da placa. Após um *squeeze* na dimensão vertical, o tensor resultante tem dimensão $(B, 256, 7)$, onde cada uma das 7 posições contém um vetor de 256 *features* que representa um caractere. Essa operação é a peça central da arquitetura: ela realiza implicitamente a separação dos caracteres, dispensando qualquer algoritmo explícito de segmentação.

- **Cabeçalhos de classificação:**

Os 7 vetores de *features* são então processados por 7 cabeçalhos independentes (*heads*), um para cada posição de caractere. Cada cabeçalho é composto por uma camada *Linear*($256 \rightarrow 128$), seguida de ReLU, *Dropout* e uma segunda camada *Linear*($128 \rightarrow C_i$), onde C_i é o número de classes daquela posição: 26 para posições de letra e 10 para posições de dígito. Essa especialização por posição elimina previsões estruturalmente inválidas (como uma letra em posição de dígito) e reduz o espaço de busca de cada cabeçalho individualmente.

A figura 33 apresenta o diagrama da arquitetura completa da rede.

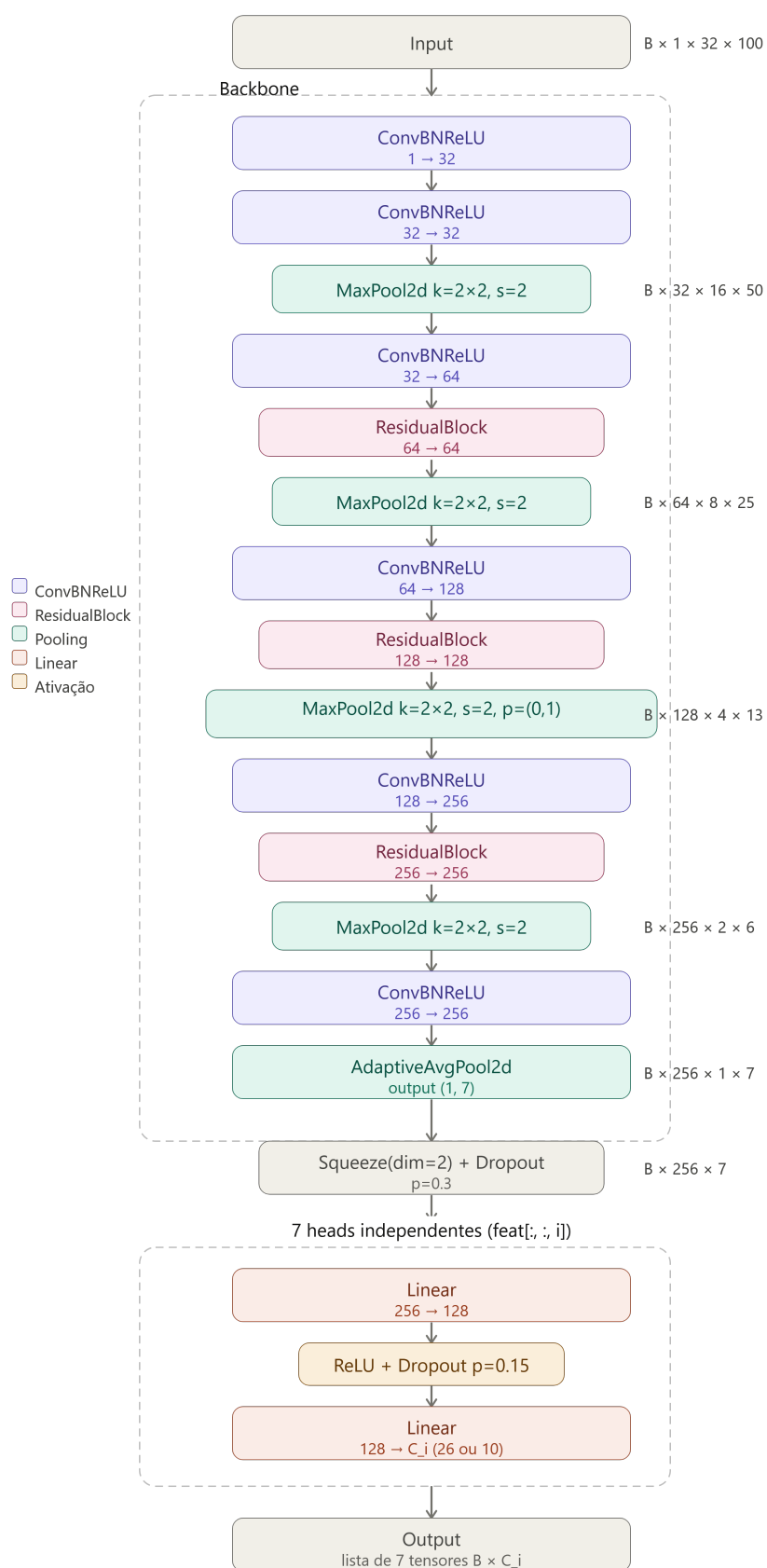


Figura 33 – Diagrama da arquitetura completa da rede de extração e classificação simultânea

Esse modelo, por ser maior e mais complexo, utilizou também algumas técnicas mais refinadas de treinamento. Uma delas foi o *learning rate* variável, que aumenta ou diminui ao longo do treinamento. Outra foi o *early stopping*, que interrompe o treinamento ao identificar que a performance está estagnada.

Os parâmetros de treinamento foram:

- *Optimizer*: AdamW com *learning rate* de 0.003 e *weight decay* de 0.004
- Tamanho do *batch*: 64
- Número de épocas: 100
- Paciência: 20

A paciência determina a quantidade de épocas que podem ocorrer sem melhoras na acurácia do modelo. Após essa quantidade de épocas sem melhoria o treinamento é interrompido. Foram treinados dois modelos distintos, um para placas do tipo Brasil e outro para placas do tipo Mercosul. As figuras 34 e 35 mostram a evolução da *Loss*, da acurácia por caractere e da acurácia por placa ao longo das épocas de treinamento.

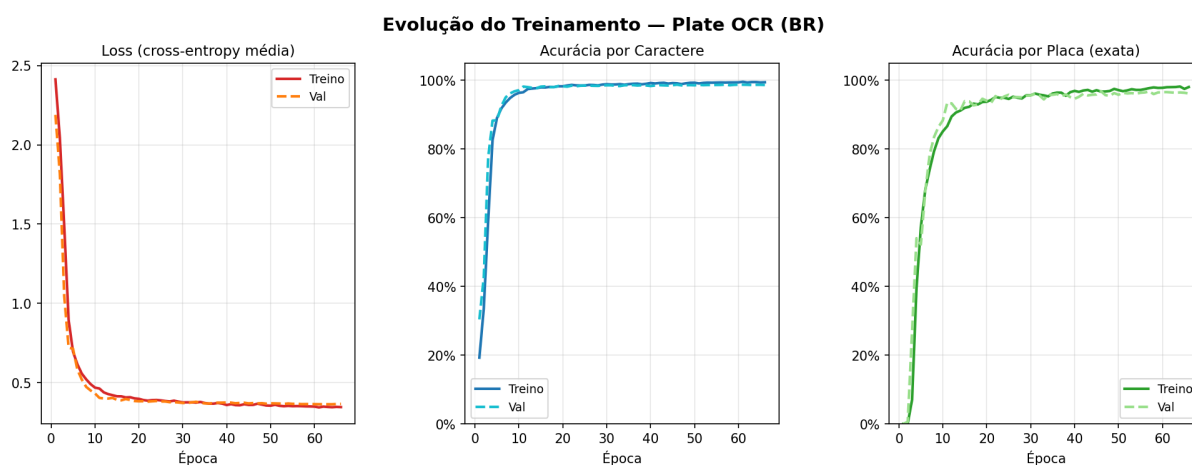


Figura 34 – Curvas de treinamento do modelo de classificação em placas do tipo Brasil

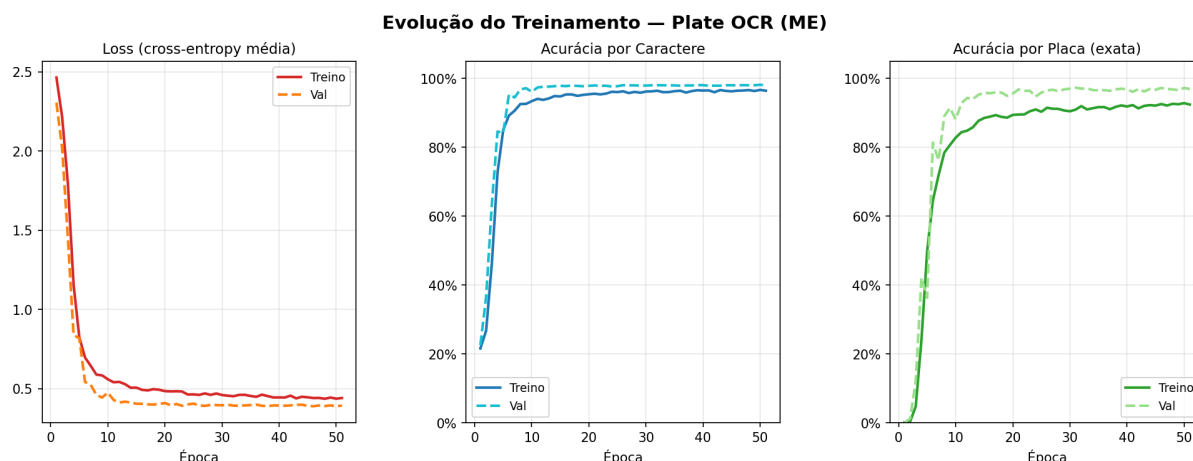


Figura 35 – Curvas de treinamento do modelo de classificação em placas do tipo Mercosul

5.6 TRANSFORMAÇÃO DOS MODELOS PARA FORMATO COMPATÍVEL COM O *HARDWARE*

Todos os modelos treinados foram convertidos para o formato ONNX, que pode ser utilizado para execução na placa Rock 3A, porém sem tirar proveito da NPU. Para que a NPU possa ser utilizada, é necessário que os modelos sejam convertidos para o formato RKNN. A Rockchip, fabricante do chip de processamento utilizado na Rock 3A, disponibiliza um *software* na forma de uma biblioteca do python que permite a compilação dos modelos nesse formato. Porém, é necessário que o programa seja executado em um ambiente *linux* da distribuição Ubuntu versão 24.

É possível, no entanto, criar um ambiente com essas condições dentro do *windows* ou outros sistemas operacionais através de um *container* Docker. Um *container* Docker é um ambiente isolado dentro de um computador no qual pode ser instalado *software* sem afetar a máquina que contém o *container*. Esse *container* pode ter qualquer sistema operacional compatível com a máquina, incluindo o Ubuntu 24. Foi então criado um *container* no qual foi instalado o python e todas as bibliotecas necessárias, incluindo o *rknn toolkit*, responsável pela conversão.

O código de conversão é um *script* simples, que necessita apenas que se aponte para o arquivo ONNX de origem, se preencha algumas informações de configuração e aponte para o *dataset* de quantização, caso se queira quantizar o modelo. A quantização tem potencial de aumentar consideravelmente a performance na NPU, sendo portanto altamente recomendada. As configurações que devem ser informadas são basicamente o modelo do chip, o nível de otimização, o algoritmo de quantização e os valores de normalização dos dados.

Como as imagens são carregadas como matrizes de números inteiros de 8 *bits*, podem ser necessárias transformações de normalização nas imagens. No modelo

YOLO, por exemplo, os pixels devem ser convertidos para valores reais de 0 a 1. Portanto, é necessário dividir os valores de 0 a 255 iniciais por 255. Isso deve ser informado para que a quantização funcione.

5.6.1 Resultados da conversão

O sucesso da conversão variou conforme o modelo:

- YOLO: sucesso na conversão porém sem quantização
- Modelo de caracteres isolados: sucesso na conversão e na quantização
- Modelo de extração e classificação: falha na conversão

Não foi possível identificar o que causou a falha na quantização do YOLO e na conversão dos modelos de extração e classificação, tendo sido tentadas várias abordagens. No caso do YOLO, a quantização é finalizada e o modelo é exportado, mas ao executar na placa os resultados saíam errados, já no caso do modelo de classificação e extração a conversão nem concluiu. A conclusão que se chegou a respeito destes erros é que algo relativo à arquitetura dos modelos não é suportado, embora não se saiba exatamente o que.

Apesar dessas falhas, a execução na placa ainda é possível com todos os modelos. Apenas não sendo possível executar na NPU o modelo que não foi convertido, sendo necessário o uso da CPU. Já o YOLO pode ser executado na NPU normalmente, perdendo apenas os ganhos de performance que a quantização poderia proporcionar.

5.6.2 Datasets de quantização

Os *datasets* de quantização foram criados separando imagens do conjunto de treinamento de cada modelo, colocando em uma pasta separada e criando um arquivo de texto contendo o caminho de cada imagem. Para a quantização bastava apontar o caminho para este arquivo que o *rknn toolkit* se encarregava do resto do processo. No caso do YOLO, os dados de quantização são imagens de carros retiradas do conjunto de treino desse modelo, enquanto para os modelos de classificação são utilizadas imagens dos respectivos *datasets* de caracteres ou de placas criados anteriormente.

5.7 PREPARAÇÃO DO HARDWARE

A preparação da placa Rock 3A para execução do algoritmo envolveu os seguintes passos de configuração:

1. Download do sistema operacional no computador
2. Gravação da imagem do sistema operacional em um cartão SD

3. Conexão de periféricos e *boot* do dispositivo
4. Configuração de ssh
5. Habilitação da NPU
6. Instalação das bibliotecas necessárias
7. Download do código, modelos e dados de teste para o dispositivo

5.7.1 Sistema operacional

A documentação do dispositivo Rock 3A inclui *links* onde é possível fazer *download* de algumas opções de imagens de sistemas operacionais que podem ser usados no Rock 3A. Essas imagens são arquivos que contêm todo o conteúdo necessário para a instalação do sistema operacional no dispositivo. A imagem escolhida foi a Debian, pois é uma distribuição do *linux* amplamente utilizada e com muito suporte. Após fazer o *download* foi necessário descompactar o arquivo, para então gravá-lo em um cartão SD.

Foi utilizado um cartão SD com 32 GB de capacidade. Para gravar a imagem foi necessário conectá-lo ao computador utilizando um adaptador para porta USB e instalar um *software* chamado Balena Etcher, indicado na própria documentação do Rock 3A. Esse software realiza a instalação do sistema operacional no cartão SD com base no arquivo de imagem selecionado.

5.7.2 Conexão de periféricos e *boot* do dispositivo

Ao inserir no dispositivo o cartão SD com o sistema operacional gravado, bastou conectá-lo a uma fonte para que o dispositivo ligasse e fizesse *boot* do sistema operacional. Porém, para que fosse possível interagir com o dispositivo foi necessário conectar periféricos que realizem a interface humano-máquina: Um monitor, um *mouse* e um teclado. Como a placa possui 3 entradas USB não foi problema conectar estes periféricos. Dessa forma passou a ser possível visualizar a área de trabalho no monitor, controlar o cursor e digitar comandos.

5.7.3 Configuração de SSH

Durante o desenvolvimento é conveniente poder acessar o Rock 3A a partir do PC de trabalho, sem precisar utilizar os periféricos conectados a ele. Para que isso fosse possível, foi configurado um servidor SSH no dispositivo. O sistema operacional já veio com o *software* necessário, de forma que só foi preciso alterar um arquivo de configuração para habilitar a conexão ssh. Além disso, foi necessário conectar o

dispositivo à rede da empresa através de um cabo de rede e descobrir o endereço IP do dispositivo dentro da rede, que estava na documentação.

Com isso, o SSH estava configurado no dispositivo. Para conectar, foi instalado no PC de trabalho um *software* de cliente SSH, que permite esse tipo de conexão e oferece uma boa interface para isso, o MobaXterm. Então, bastou conectar o PC à rede da empresa (mesma rede na qual o Rock 3A estava conectado) e na interface do MobaXterm criar uma nova conexão, na qual foi informado o IP do dispositivo, o nome do usuário e a senha. Finalizado isso, o MobaXterm abriu um terminal no PC onde era possível digitar qualquer comando, que seria então enviado e executado no dispositivo.

Com isso configurado e funcionando, os periféricos puderam ser desconectados do dispositivo, mantendo apenas conectado o cabo de rede, pois qualquer comando poderia agora ser enviado remotamente através da conexão ssh criada.

5.7.4 Habilitação da NPU e instalação das bibliotecas

A NPU não vem habilitada por padrão, sendo necessário acessar um menu de configuração do Rock 3A para habilitá-la. Esse menu é acessado através do comando *rsetup*, que leva para uma tela de configurações onde deve ser selecionada a opção "*overlays*" e "*manage overlays*", depois navegar para a opção "NPU" e ativar. Por fim, é necessário reiniciar o dispositivo para que a configuração surta efeito.

O python já vem instalado no sistema operacional, mas é necessário instalar algumas bibliotecas para que o algoritmo desenvolvido possa ser executado. As bibliotecas básicas são:

- OpenCV
- Numpy
- Pillow
- Scikit-Image
- Onnx
- Onnx Runtime

Elas permitem carregamento e manipulação de imagens, manipulação de vetores e matrizes, execução de modelos no formato ONNX e execução de todas as funções necessárias para as etapas do *pipeline* de leitura de placas desenvolvido. Com essas bibliotecas já é possível executar a solução com os modelos na CPU, e todas podem ser instaladas através de um simples comando "*pip install <nome>*".

Para execução dos modelos na NPU, porém, é necessário instalar a biblioteca RKNN Toolkit Light. Essa difere do RKNN Toolkit tradicional pois serve apenas para

executar modelos RKNN, não para realizar a conversão dos modelos para esse formato. Essa biblioteca possui um processo de instalação menos automático, sendo necessário acessar um repositório no *github* onde ela se encontra e baixar um arquivo do tipo *wheel* que corresponda à versão do python sendo utilizada (python 3.10). Com isso, basta executar o comando "*pip install <arquivo>*", passando o caminho para o arquivo *wheel* que foi baixado.

Para finalizar, o RKNN Toolkit ainda necessita que um arquivo adicional chamado *librknn.so* seja baixado do repositório e colocado na pasta */usr/lib* do sistema de arquivos.

5.7.5 Download do código para o dispositivo

Através da interface do MobaXterm conectado ao Rock 3A é possível navegar pelas pastas do dispositivo e fazer *download* de arquivos do PC para estas pastas. Foi então feito *download* dos seguintes itens:

- Código python contendo o *pipeline* completo de leitura de placas, que recebe uma imagem e devolve os caracteres
- *Script* de teste, que compara a placa devolvida pelo *pipeline* com a placa verdadeira para todas as imagens do conjunto de teste e computa os tempos de processamento
- Conjunto de dados de teste, composto por imagens de carros e os caracteres da placa correspondente
- Modelo YOLO, modelos de classificação de caracteres isolados e modelos de extração e classificação no formato ONNX
- Modelo YOLO e modelos de classificação de caracteres isolados no formato RKNN

5.8 TESTES REALIZADOS

Para realizar os testes do algoritmo completo, foi separado um conjunto de imagens do *dataset* de teste separado inicialmente. O *dataset* continha 1000 imagens, mas como algumas eram do mesmo carro, foram mantidas 849. As imagens foram colocadas em uma pasta separada e renomeadas, para que o nome da imagem fosse exatamente a placa contida no carro. Foi feito então um *script* simples que pega cada uma dessas imagens, envia ao *pipeline* desenvolvido de leitura de placas e compara o resultado obtido com a placa verdadeira, verificando assim se foi um acerto ou um erro. O *script* também computa o tempo levado em cada etapa do *pipeline*, detecção com o modelo YOLO, processamento da imagem e classificação dos caracteres.

5.8.1 Resultados dos testes

Os testes foram realizados em seis configurações distintas, combinando diferentes *pipelines* e plataformas de *hardware*. As duas alternativas de *pipeline* avaliadas foram:

- **Pipeline de caracteres isolados:** realiza a extração dos caracteres por meio de técnicas de processamento de imagem e os classifica individualmente com os modelos de classificação de caracteres.
- **Pipeline de extração e classificação simultânea:** utiliza o modelo que recebe a imagem da placa completa e classifica todos os seus caracteres de uma só vez.

As plataformas avaliadas foram o PC (utilizando a CPU) e a placa Rock 3A, com execução tanto na CPU quanto na NPU. É importante ressaltar que, conforme discutido na seção anterior, o modelo de extração e classificação simultânea não pôde ser convertido para o formato RKNN, impossibilitando sua execução na NPU da Rock 3A. Da mesma forma, a quantização do modelo YOLO não foi bem-sucedida, de modo que nos testes na Rock 3A o YOLO é sempre executado sem quantização, independentemente do *pipeline* utilizado.

Dada essa restrição, os modelos de classificação de caracteres isolados foram os únicos convertidos e quantizados. Portanto, para o *pipeline* de caracteres isolados na NPU da Rock 3A, todos os modelos são executados na NPU, apesar do YOLO não ser quantizado. Já para o *pipeline* de extração e classificação simultânea na Rock 3A, o YOLO é executado na NPU e o modelo de extração e classificação na CPU. A Tabela 5 apresenta os resultados de acurácia obtidos em cada configuração de teste, e a Tabela 6 apresenta os tempos médios de inferência por etapa do *pipeline*. Nas tabelas o *pipeline* de caracteres isolados foi chamado apenas de "caracteres", enquanto o de extração e classificação foi chamado de "placas", fazendo referência ao tipo de *input* dos modelos de cada *pipeline*.

Tabela 5 – Acurácia por configuração de teste

Hardware	Pipeline	Acertos	Erros	Acurácia
PC (CPU)	Placas	773	76	91,05%
Rock 3A (CPU)	Placas	773	76	91,05%
Rock 3A (NPU+CPU)	Placas	773	76	91,05%
PC (CPU)	Caracteres	727	122	85,63%
Rock 3A (CPU)	Caracteres	727	122	85,63%
Rock 3A (NPU)	Caracteres	728	121	85,75%

Tabela 6 – Tempos médios de inferência por etapa (ms)

Hardware	Pipeline	Deteccção	Extração	Leitura	Total
PC (CPU)	Placas	57,24	4,40	8,84	70,48
Rock 3A (CPU)	Placas	718,93	18,05	58,64	795,63
Rock 3A (NPU+CPU)	Placas	395,81	13,10	55,20	464,10
PC (CPU)	Caracteres	151,91	21,54	22,70	196,15
Rock 3A (CPU)	Caracteres	727,39	81,99	26,93	836,31
Rock 3A (NPU)	Caracteres	422,10	86,94	30,18	539,22

Analisando os resultados de acurácia, é possível observar que o *pipeline* de extração e classificação simultânea apresentou desempenho superior ao de caracteres isolados, atingindo 91,05% de acurácia frente a 85,63% obtidos pelo segundo. Essa diferença de aproximadamente 5,4 pontos percentuais pode ser atribuída ao fato de que o modelo de extração e classificação opera sobre a imagem da placa completa, sendo menos suscetível a erros introduzidos pelo algoritmo de segmentação de caracteres, que representa uma etapa adicional de processamento no *pipeline* de caracteres isolados.

Outro resultado relevante diz respeito à acurácia obtida com o *pipeline* de caracteres isolados quantizado, executado na NPU da Rock 3A. A acurácia de 85,75% foi praticamente idêntica à obtida pela versão não quantizada (85,63%), demonstrando que o processo de quantização não introduziu perda perceptível de desempenho nos modelos de classificação, sendo portanto uma estratégia válida para ganho de eficiência computacional sem prejuízo à confiabilidade do sistema. A acurácia observada é consistente entre o PC e a Rock 3A para um mesmo *pipeline* e mesmos modelos, o que era esperado, pois a plataforma de *hardware* não interfere nos pesos dos modelos e, portanto, não altera os resultados das inferências.

No que diz respeito aos tempos de inferência, o PC apresentou desempenho significativamente superior à Rock 3A em todos os cenários. O *pipeline* de extração e classificação simultânea executado no PC foi o mais rápido, com tempo médio total de 70,48 ms, enquanto o mesmo *pipeline* na Rock 3A com CPU atingiu 795,63 ms, um aumento de aproximadamente 11 vezes. Já o *pipeline* de caracteres isolados no PC totalizou 196,15 ms, contra 836,31 ms na Rock 3A com CPU, representando um fator de aproximadamente 4,3 vezes.

A comparação entre os dois *pipelines* no PC revela uma diferença expressiva: o *pipeline* de extração e classificação simultânea é cerca de 2,8 vezes mais rápido. Isso ocorre pois, além de eliminar o algoritmo de segmentação de caracteres, o modelo de extração e classificação realiza uma única inferência para toda a placa, enquanto o *pipeline* de caracteres isolados realiza múltiplas inferências, uma por caractere. Na Rock 3A, contudo, a diferença entre os dois *pipelines* executados inteiramente na CPU foi menos acentuada, de apenas 40 ms aproximadamente, possivelmente em

razão das diferenças na eficiência de execução dos modelos na arquitetura ARM do dispositivo.

O uso da NPU na Rock 3A trouxe ganhos expressivos de velocidade em ambos os *pipelines*. No *pipeline* de caracteres isolados, a execução com todos os modelos na NPU, ainda que o YOLO sem quantização, resultou em uma redução de aproximadamente 35% no tempo total em comparação com a execução inteiramente na CPU da mesma placa (539,22 ms contra 836,31 ms). Essa melhora se deve principalmente à aceleração da etapa de detecção, que passou de 727,39 ms na CPU para 422,10 ms na NPU, uma redução de cerca de 42%.

Para o *pipeline* de extração e classificação simultânea, a utilização da NPU para a execução do YOLO, mantendo o modelo de extração e classificação na CPU, resultou no melhor desempenho entre as configurações testadas na Rock 3A, com tempo total de 464,10 ms. Isso representa uma redução de aproximadamente 42% em relação à versão com todos os modelos na CPU (795,63 ms). A etapa de detecção caiu de 718,93 ms para 395,81 ms, evidenciando que a aceleração do YOLO na NPU é o principal fator de ganho de desempenho nesse cenário.

A Tabela 7 sintetiza a comparação entre os cenários avaliados, ordenados pelo tempo total de inferência, destacando a relação entre acurácia e velocidade de cada configuração.

Tabela 7 – Comparativo geral entre configurações de teste

Hardware	Pipeline	Acurácia	Total (ms)
PC (CPU)	Placas	91,05%	70,48
Rock 3A (CPU)	Placas	91,05%	795,63
Rock 3A (NPU+CPU)	Placas	91,05%	464,10
PC (CPU)	Caracteres	85,63%	196,15
Rock 3A (CPU)	Caracteres	85,63%	836,31
Rock 3A (NPU)	Caracteres	85,75%	539,22

De maneira geral, os resultados demonstram que o *pipeline* de extração e classificação simultânea se mostrou superior tanto em acurácia quanto em velocidade no PC, sendo a configuração mais eficiente entre as avaliadas. Na Rock 3A, a combinação do YOLO na NPU com o modelo de extração e classificação na CPU entregou o melhor desempenho entre as opções do dispositivo, com 464,10 ms e acurácia de 91,05%. Essa configuração supera também o *pipeline* de caracteres isolados com quantização (539,22 ms e 85,75%), que embora mais lento, representa a melhor alternativa quando apenas os modelos de classificação estão disponíveis no formato RKNN.

6 CONCLUSÃO

Neste capítulo são apresentadas as considerações finais do trabalho, sintetizando o problema abordado, a solução proposta e os principais resultados obtidos. Em seguida, discute-se o alcance dos objetivos inicialmente traçados, as limitações encontradas, possíveis aprimoramentos e os impactos do projeto. Por fim, são sugeridos trabalhos futuros que podem dar continuidade ao desenvolvimento realizado.

6.1 SÍNTESE DO PROBLEMA, SOLUÇÃO E RESULTADOS

O presente trabalho tratou do desenvolvimento de um sistema de leitura automática de placas de veículos para aplicações de controle de acesso, com foco em viabilizar uma solução de baixo custo executável em hardware acessível. O problema surgiu no contexto do produto *Aegis Access Control* da Palmsoft Tecnologia, onde o alto custo das câmeras LPR comerciais (como o modelo VIP 5460 LPR IA da Intelbras) motivou a busca por alternativas econômicas, especialmente para condomínios autônomos de aluguel de curto prazo.

A solução proposta consistiu em um *pipeline* de visão computacional composto por três etapas principais: Detecção e classificação da placa utilizando o modelo YOLO26n-pose, processamento da imagem da placa (conversão para escala de cinza, correção de perspectiva, remoção de bordas, binarização adaptativa com método de Sauvola, segmentação e limpeza de caracteres) e classificação dos caracteres extraídos por meio de redes neurais convolucionais especializadas. Paralelamente, foi desenvolvida uma abordagem alternativa baseada em modelos de extração e classificação simultânea dos caracteres diretamente da imagem da placa.

O desenvolvimento seguiu o padrão CRISP-DM e utilizou o *dataset* RodoSol-ALPR. Os modelos foram treinados, exportados para ONNX e, quando possível, convertidos para o formato RKNN com quantização *uint8* para execução na NPU da placa Radxa ROCK 3A (Rockchip RK3568).

Os principais resultados obtidos foram:

- Acurácia de 91,05% com o *pipeline* de extração e classificação simultânea e 85,63–85,75% com o *pipeline* de caracteres isolados;
- Tempo de inferência no PC (CPU) de 70,48 ms (pipeline simultâneo) e 196,15 ms (caracteres isolados);
- Na ROCK 3A, o melhor desempenho foi alcançado com YOLO na NPU + modelo simultâneo na CPU (464,10 ms) e com todos os modelos do pipeline de caracteres isolados na NPU (539,22 ms).

Esses resultados demonstram a viabilidade técnica da solução tanto em termos de precisão quanto de performance em hardware de baixo custo.

6.2 OBJETIVOS ATINGIDOS E LIMITAÇÕES

O trabalho cumpriu integralmente o objetivo geral de desenvolver um software de leitura automática de placas de veículos e validar a viabilidade funcional de sua execução na placa ROCK 3A. Foram atingidos todos os requisitos funcionais e não funcionais definidos, incluindo suporte aos dois padrões de placas brasileiras (Brasil e Mercosul), detecção da classe da placa, acurácia acima de 85% e execução otimizada na NPU sempre que possível.

A placa ROCK 3A demonstrou ser capaz de executar o sistema de forma satisfatória, com redução significativa de tempo de inferência quando se utiliza a NPU (ganhos de até 42% na etapa de detecção). A quantização dos modelos de classificação de caracteres não gerou perda perceptível de acurácia, confirmando sua adequação para aplicações embarcadas.

Entre as limitações encontradas destacam-se:

- Dificuldades na conversão para RKNN de alguns modelos (especialmente o de extração e classificação simultânea e a quantização completa do YOLO), possivelmente por incompatibilidades de operações com o toolkit da Rockchip;
- Tempo de inferência ainda elevado na ROCK 3A para aplicações de tempo real extremamente exigentes (embora adequado para a maioria dos cenários de controle de acesso);
- Dependência de condições de iluminação e ângulo favoráveis, comum a sistemas de visão computacional baseados em imagens estáticas.

6.3 APRIMORAMENTOS POSSÍVEIS

Diversas melhorias podem ser implementadas para elevar o desempenho e a robustez do sistema. Entre elas:

- Aperfeiçoamento do algoritmo de segmentação de caracteres e experimentação de novas técnicas de binarização e filtragem de ruído;
- Treinamento com maior volume de dados ou aplicação de técnicas de aumento de dados (data augmentation) para aumentar a robustez a variações de iluminação, sujeira e ângulos extremos;
- Exploração de modelos mais recentes ou destilação de conhecimento para reduzir ainda mais o tamanho e latência dos modelos sem perda de acurácia;

- Implementação de tracking temporal em vídeo para aumentar a confiabilidade através da combinação de múltiplos frames;
- Otimização adicional da conversão para RKNN, possivelmente com suporte técnico da Rockchip ou uso de versões mais recentes do toolkit.

6.4 IMPACTOS DO PROJETO

O projeto gera impactos positivos significativos para a Palmsoft Tecnologia e seus clientes. Do ponto de vista tecnológico, demonstra a viabilidade de soluções de IA em hardware de baixo custo, ampliando o portfólio da empresa em visão computacional e controle de acesso. Financeiramente, a redução drástica no custo por unidade (da ordem de R\$ 5.700–7.500 para valores próximos a algumas centenas de reais) torna o sistema acessível a condomínios menores, estacionamentos de médio porte e outras aplicações anteriormente inviáveis economicamente.

Para os clientes finais (gestores de condomínios e estabelecimentos), o sistema proporciona maior comodidade, agilidade na entrada e saída de veículos e redução de custos operacionais. Organizacionalmente, valida o uso de placas como a ROCK 3A em produtos da empresa, podendo acelerar o desenvolvimento de novas soluções de IA embarcadas.

6.5 TRABALHOS FUTUROS

O desenvolvimento realizado abre diversas possibilidades de continuidade. Sugere-se:

- Construção de um protótipo completo de câmera LPR utilizando a ROCK 3A (ou versão superior), incluindo integração com o *Aegis Access Control*;
- Expansão do sistema para leitura de placas de motocicletas e outros veículos;
- Desenvolvimento de uma versão otimizada para múltiplas câmeras ou processamento em borda com agregação em nuvem;
- Aplicação da mesma arquitetura em outros problemas de visão computacional, como contagem de veículos, detecção de irregularidades ou monitoramento de tráfego;
- Publicação de partes do código e modelos como open-source, contribuindo para a comunidade de pesquisa em ALPR no Brasil.

Em suma, o presente Projeto Final de Curso demonstrou não apenas a viabilidade técnica e econômica da proposta, mas também pavimentou o caminho para o

desenvolvimento de produtos inovadores e acessíveis na área de controle de acesso veicular.

REFERÊNCIAS

CASEY, Richard G.; LECOLINET, Eric. A Survey of Methods and Strategies in Character Segmentation. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, IEEE, v. 18, n. 7, p. 690–706, 1996. Revisão abrangente de técnicas de segmentação, incluindo histograma de projeção vertical.

CHAPMAN, Peter; CLINTON, Julian; KERBER, Randy; KHABAZA, Thomas; REINARTZ, Thomas; SHEARER, Colin; WIRTH, Rüdiger. **CRISP-DM 1.0: Step-by-step data mining guide**. [S.l.], 2000. Technical report. Disponível em: <https://www.crisp-dm.org>.

CHEN, Yunji; XIE, Yuan; SONG, Linghao; CHEN, Fan; TANG, Tianshi. A Survey of Accelerator Architectures for Deep Neural Networks. **Engineering**, Elsevier, v. 6, n. 3, p. 264–274, 2020. Panorama de aceleradores de DNN incluindo NPUs embarcadas como as encontradas em SoCs móveis e de borda.

FAYYAD, Usama M.; PIATETSKY-SHAPIRO, Gregory; SMYTH, Padhraic. From Data Mining to Knowledge Discovery in Databases. **AI Magazine**, v. 17, n. 3, p. 37–54, 1996. DOI: 10.1609/aimag.v17i3.1230. Disponível em: <https://ojs.aaai.org/index.php/aimagazine/article/view/1230>.

IBM INC. **What are convolutional neural networks?** [S.l.: s.n.], 2024. <https://www.ibm.com/>. Acessado em março de 2026. Disponível em: <https://www.ibm.com/think/topics/convolutional-neural-networks>.

JACOB, Benoit; KLIGYS, Skirmantas; CHEN, Bo; ZHU, Menglong; TANG, Matthew; HOWARD, Andrew; ADAM, Hartwig; KALENICHENKO, Dmitry. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, p. 2704–2713, 2018. Artigo seminal sobre quantização int8 com zero-point; base do esquema usado no TFLite e no RKNN toolkit.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. ImageNet Classification with Deep Convolutional Neural Networks. In: **ADVANCES in Neural Information Processing Systems 25 (NIPS 2012)**. [S.l.: s.n.], 2012. p. 1097–1105. Disponível em: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>.

LAROCA, R.; CARDOSO, E. V.; LUCIO, D. R.; ESTEVAM, V.; MENOTTI, D. On the Cross-Dataset Generalization in License Plate Recognition. In: **INTERNATIONAL Conference on Computer Vision Theory and Applications (VISAPP)**. [S.l.: s.n.], fev. 2022. p. 166–178. DOI: 10.5220/0010846800003124.

LECUN, Yann; BOTTOU, Léon; BENGIO, Yoshua; HAFFNER, Patrick. Gradient-Based Learning Applied to Document Recognition. **Proceedings of the IEEE**, v. 86, n. 11, p. 2278–2324, 1998. DOI: 10.1109/5.726791. Disponível em: <https://doi.org/10.1109/5.726791>.

REDMON, Joseph; DIVVALA, Santosh; GIRSHICK, Ross; FARHADI, Ali. You Only Look Once: Unified, Real-Time Object Detection. In: PROCEEDINGS of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). [S.l.: s.n.], 2016. p. 779–788.

RUMELHART, David E.; HINTON, Geoffrey E.; WILLIAMS, Ronald J. Learning Representations by Back-propagating Errors. **Nature**, Nature Publishing Group, v. 323, p. 533–536, 1986. Artigo seminal que formalizou o algoritmo de backpropagation.

SAHA, Sumit. A Comprehensive Guide to Convolutional Neural Networks — the ELI5 way. **Medium**, 2018.

SAUVOLA, Jaakko; PIETIKÄINEN, Matti. Adaptive Document Image Binarization. **Pattern Recognition**, Elsevier, v. 33, n. 2, p. 225–236, 2000. Referência para binarização adaptativa local, análoga à abordagem por faixas do projeto.

THEAILEARNER. **Perspective Transformation**. [S.l.: s.n.], 2023.
<https://theailearner.com>. Acessado em março de 2026. Disponível em:
<https://theailearner.com/tag/cv2-getperspectivetransform/>.

ZOU, Zhengxia; SHI, Zhenwei; GUO, Yuhong; YE, Jieping. Object Detection in 20 Years: A Survey. **arXiv preprint arXiv:1905.05055**, 2019.

APÊNDICE A – DESCRIÇÃO 1

Textos elaborados pelo autor, a fim de completar a sua argumentação. Deve ser precedido da palavra APÊNDICE, identificada por letras maiúsculas consecutivas, travessão e pelo respectivo título. Utilizam-se letras maiúsculas dobradas quando esgotadas as letras do alfabeto.

ANEXO A – DESCRIÇÃO 2

São documentos não elaborados pelo autor que servem como fundamentação (mapas, leis, estatutos). Deve ser precedido da palavra ANEXO, identificada por letras maiúsculas consecutivas, travessão e pelo respectivo título. Utilizam-se letras maiúsculas dobradas quando esgotadas as letras do alfabeto.